

可视计算与交互概论

BAOQUAN CHEN MENGYU CHU

Organized by Elanor

2026 Fall

目录

LESSON ONE 0914早八	?
一、课程导入	?
二、显示设备的发展	?
2.1 早期显示设备：从数码管到CRT	?
2.2 CRT显示器：从矢量显示到二维光栅显示	?
2.3 彩色显示与颜色空间	?
(1) RGB颜色空间	?
(2) HSV颜色空间	?
(3) 色域与人眼可见范围	?
(4) 伽马校正与颜色位深	?
2.4 LCD液晶显示器	?
2.5 OLED有机发光显示器	?
2.6 色域标准	?
2.7 分辨率的发展	?
2.8 立体显示技术展望	?
(1) 双目视觉原理	?
(2) 偏振3D与头显技术	?
(3) AR/VR/MR	?
(4) 裸眼3D与全息显示	?
三、色彩在可视化中的应用	?
四、图像的定义与表达	?
4.1 图像的数学定义：连续与离散	?
4.2 矢量图与光栅图	?
(1) 矢量图	?
(2) 光栅图（位图）	?
(3) 两者的对比	?
五、帧缓冲区与GPU基础	?
5.1 GPU的诞生与图形管线	?
5.2 帧缓冲区（Frame Buffer）	?
5.3 图像存储格式	?
(1) 调色板格式（GIF）	?
(2) 其他常见格式	?
5.4 Alpha通道与图像合成	?
5.5 深度缓冲（Z-Buffer）	?
5.6 双缓冲区	?
六、图像处理概述与课程预告	?

课程预告	?
LESSON 2 0915早十	?
图像处理：抖动、量化、滤波、边缘提取、补全与抠图	?
一、图像处理引入	?
二、抖动与量化	?
1. 背景	?
2. 直接二值化的问题	?
3. 抖动的思想	?
三、白噪声与蓝噪声	?
四、滤波与模糊	?
五、有序抖动与空间换颜色	?
六、Floyd-Steinberg 抖动	?
七、图像滤波与卷积 <i>image filtering</i>	?
八、边缘提取、Sobel 与拉普拉斯	?
九、频率的直观理解	?
十、图像补全与拉普拉斯编辑	?
十一、图像克隆、融合与泊松编辑	?
十二、泊松方程的离散求解	?
十三、抠图	?
LESSON 3 2026-09-21 第1-2节	?
一、课程导入与内容安排	?
二、光栅化绘制的背景与核心需求	?
2.1 显示硬件的演进：从矢量到光栅	?
2.2 帧缓冲区与光栅化流程	?
2.3 画线操作的普遍性与性能要求	?
2.4 扫描绘制的核心目标	?
三、直线绘制的基础算法	?
3.1 暴力算法及其缺陷	?
3.2 朴素迭代算法	?
3.3 朴素算法的性能分析	?
四、DDA数字微分分析仪算法	?
4.1 优化思路：乘法转加法迭代	?
4.2 算法步骤	?

4.3 仍然存在的问题	?
五、布雷森汉姆 (Bresenham) 直线算法	?
5.1 核心观察：像素步进的二选一	?
5.2 中点判定法与隐函数思想	?
5.3 构造整数隐函数	?
5.4 判别式的迭代更新	?
5.5 算法推导与迭代过程	?
5.6 算法伪代码	?
5.7 算法意义	?
六、直线算法的扩展	?
6.1 全空间直线的八分区处理	?
6.2 扩展到画圆	?
6.3 多边形扫描线算法	?
6.4 凹多边形与三角化	?
七、颜色插值与图像变形	?
7.1 颜色线性插值	?
7.2 双线性插值	?
7.3 图像变形 (Image Warping)	?
7.4 交互式形变的实现	?
7.5 四边形插值的两种方式	?
八、曲线内容展望	?
8.1 曲线的表达需求	?
8.2 矢量图与曲线	?
8.3 工业领域的曲线应用：CAD/CAE/CAM	?
8.4 后续课程预告	?
LESSON 4 2026-09-22 第5-6节	?
课程导入	?
显式曲线	?
隐式曲线	?
参数化曲线	?
参数化曲线的定义与优势	?
多项式曲线及其不足	?
分段曲线的思路	?
插值与基函数基础	?
线性插值	?
基函数的概念	?
贝塞尔曲线	?
直观认识与构造过程	?
阶数与递归构造	?
代码实现思路	?
数学形式与伯恩斯坦基函数	?

数值稳定性与矩阵形式	?
端点与切线性质	?
连续性: C连续与G连续	?
贝塞尔曲面 (三维扩展)	?
凸包性与局限性	?
前沿拓展: 犹他茶壶与G2连续问题	?
其他参数化曲线	?
三次样条曲线	?
Catmull-Rom曲线与厄米曲线	?
Catmull-Rom曲线	?
厄米曲线	?
B样条曲线简介	?
课堂问答	?
问题1: C1连续和G1连续的核心区别是什么?	?
问题2: 厄米曲线和贝塞尔曲线的对应关系是什么?	?
问题3: 伯恩斯坦多项式可以表示任意多项式曲线吗?	?
问题4: 考试中曲线部分的考察重点是什么?	?

LESSON ONE 0914早八

一、课程导入

同学们晚上好，接下来我们继续可视计算的课程。今天我们要讲的内容依然是比较基础的部分，从大家日常接触最多的显示设备入手，带领大家感受从显示到图像、再到可视计算的完整脉络。感谢大家来赶早八的课程，早八到课的同学不多，大家可以尽量往前坐。

今天我们围绕显示展开，核心内容是显示硬件的基础发展历程。
在计算机图形学与可视计算领域，软硬件协同发展是非常鲜明的学科特点。我们会从最基础的显示硬件讲起，逐步介绍GPU、CPU等计算单元，以及与硬件协同发展的算法、数据结构与逻辑，这些都是可视计算领域的核心内容。
这一页展示的是历史上多款商业化一体式台式机的演进流程。

二、显示设备的发展

2.1 早期显示设备：从数码管到CRT

其中比较早的是1998年的背投式设备，和早期电视类似，采用的是我们稍后会讲到的CRT（阴极射线管）显示技术。

我个人很喜欢2002年发布的台灯式iMac（iMac G4），这是一款非常经典的产品。它是第一款商业化的台式一体机中首次使用LCD屏幕的机型，让屏幕厚度大幅降低，在当时属于高端个人台式机。

等到后续课程讲到人机交互时，我们会更细致地讲解各类交互设备，届时还可以再回顾这款产品。它在当年是兼具设计感与品味的产品，而后来的显示器也沿着越来越薄的方向持续发展，直到今天的形态。

我们先简单梳理显示设备的发展脉络，大家可能对各类硬件已经有一定了解。

最早的简单显示设备是LED数码管，现在依然常见于红绿灯倒计时、嵌入式设备的数字显示等场景。它的特点是控制简单、亮度高、成本低廉，非常适合稀疏的数字与符号显示。但对于图形学而言，它的颗粒度和分辨率远不足以满足视觉显示的需求。

在此之后，显示技术逐步发展，先是出现了厚重的阴极射线管（CRT）显示器，后续又诞生了液晶显示器、等离子显示器等。随着技术进一步发展，为了满足更多前沿交互需求，各类新型显示设备也在持续研发中。以上是显示设备发展的整体概览。

2.2 CRT显示器：从矢量显示到二维光栅显示

接下来我们详细讲解CRT显示器。它曾广泛应用于电视和个人电脑，现在已经属于历史产物。同学们可能未必都用过，但在物理实验中接触过的示波器，就属于同类的阴极射线显示设备。

这类设备的核心是电子枪，它发射电子束，电子束的方向通过电极控制，最终打在前方涂有荧光层的屏幕上，使固定区域被点亮。因为电子带负电，这类射线被称为阴极射线，这类设备也因此得名。

早期CRT显示设备的电子束发射与位置控制频率不高，因此主要应用于矢量显示场景，比如示波器和简单显示单元。它的工作方式类似矢量作图：让电子束按照固定顺序移动，在屏幕上画出线条；快速遍历所有需要点亮的笔画，依靠人眼的视觉暂留和荧光粉的余晖，在视觉暂留时间内呈现完整的画面，满足稀疏内容的显示需求，同时功耗和亮度都相对可控。

原理图中可以看到，我们通过控制电场，让电子束做规则运动，在固定周期内系统性地刷新屏幕，最终呈现出矢量形式的内容。

在当时，硬件会接收算法下发的指令，比如“从某一坐标到另一坐标画一条线段”，硬件底层完成电场控制与划线操作，这是软硬件高度协同的体现。后续课程中我们会讲解直线生成的算法，也会穿插相关的交互实践。

今天我们的核心是显示技术，矢量显示只是其中的一个发展阶段。从矢量显示出发，技术逐步演进到二维光栅显示。

所谓二维显示，就是面向完整屏幕的显示。它的实现逻辑很直接：既然可以控制电子束划线，那就用扫描线对整个屏幕从左到右、从上到下完成完整扫描，所有需要点亮的位置都按照对应亮度点亮。

这是一帧扫描线执行过程的慢速演示。在实际应用中，整个扫描过程会在1/60秒内完成，依靠视觉暂留形成连续画面。早期CRT显示器的分辨率并不高，光栅扫描的性能主要受限于电子束的刷新频率，最初的显示刷新率大约只有24Hz。

从矢量显示到二维显示的演进，催生了一系列影响整个可视计算领域的核心概念：

- **像素**：屏幕上每次被点亮的最小单位区域，就是我们常说的像素。每个像素的亮暗状态是显示的基本单元。

- **分辨率**：由硬件能力决定，指在指定刷新率下屏幕能够刷新的像素总数，是屏幕显示能力的核心指标。
- **纵横比**：屏幕的宽高比例，是符合人眼视觉舒适度的设计参数。
- **帧与帧率**：完整扫描完一整幅画面称为一帧，每秒刷新的帧数就是帧率。

早期逐行扫描的刷新率较低时，人眼可能会感知到扫描的先后顺序。为了解决这个问题，人们发明了隔行扫描算法，在控制电子束移动距离的同时高效完成全屏刷新，同时保持屏幕亮度均衡。这些由硬件发展催生的概念，对后续的软件技术和整个行业都产生了深远影响。

2.3 彩色显示与颜色空间

在灰度二维显示技术成熟之后，人们在屏幕表面涂上红、绿、蓝三种不同的荧光粉涂层，实现了彩色荧光屏。

上节课我们已经讲过颜色的相关原理：对于人眼而言，只要最终接收到的红、绿、蓝三种光的强度相同，无论它来自连续光谱还是离散光谱，人感知到的颜色都是一致的。人类视觉只关心三种颜色贡献的积分总和，因此我们可以用红、绿、蓝三原色的组合，呈现出人类可感知的所有彩色，这也是彩色显示技术的基础。

讲到这里，我们对上节课的颜色内容做一些补充。

(1) RGB颜色空间

上节课我们已经讲解了红绿蓝三原色，以及离散光谱与连续光谱对人眼的感知效果相近的原理。在此基础上，我们进一步扩展颜色空间的定义。

正如软硬件协同发展的规律，人们发现屏幕可以设置红、绿、蓝三个独立的荧光层，荧光层的发光强度由电子束的击打强度决定，由此可以精准控制每个像素的灰度值，进而构成完整的颜色空间。

这就是对硬件最友好、结构最简单的RGB三原色空间。当RGB三个通道的值均为1.0时，对应白色；纯红、纯绿、纯蓝分别位于RGB立方体的顶点；两色混合可以得到玫红、黄色等常见颜色；立方体内部则对应不同比例的混合色。

在RGB空间中，任意一个点都可以唯一确定红、绿、蓝三个通道的强度，硬件可以直接根据数值控制电子束的位置和强度，最终呈现对应颜色。因此早期的颜色空间设计完全服务于屏幕显示。但在实际使用中，人们发现这种对硬件友好的颜色空间，对人类用户而言可读性较差。如果没有绘画相关的经验，很难通过调整RGB三个通道的数值得到想要的颜色，数值的变化带来的视觉感受是跳跃的、不直观的。

(2) HSV颜色空间

为了定义更符合人眼感知的颜色空间，人们提出了HSV颜色空间。

HSV同样是三通道颜色模型，但它按照人类的感知习惯，将颜色拆分为三个相互独立的语义通道：

- **色调 (Hue)**：决定颜色的种类，比如红色、蓝色，只和色相本身相关。
- **饱和度 (Saturation)**：决定颜色的浓郁程度，从高饱和的鲜艳色到低饱和的灰调。
- **明度 (Value)**：决定颜色的明暗程度，从纯黑到纯白。

拆分后每个通道都有独立的语义，更便于用户控制，因此应用非常广泛。比如Office软件中的调色板，上方的滑块调节的就是色调，选中色调后下方的明暗区域调节的就是明度，这就是HSV模型的典型应用。

HSV的另一个优势是，它与RGB空间可以通过简单的线性变换实现一一映射。这意味着它既拥有人机交互友好的调节界面，又可以精准映射到硬件适配的RGB颜色空间，很好地解决了颜色空间的控制问题。

需要注意的是，HSV空间并不是正交的立体空间。在RGB空间中，坐标不同就对应不同的颜色；但在HSV空间中，同一种颜色可能对应多个坐标点。

举例来说，当饱和度趋近于0时，颜色呈现黑白灰的无彩色状态，此时无论色调是多少，最亮时都是白色，最暗时都是黑色，也就是空间的中心轴是完全重合的。因此HSV空间可视化后并不是立方体，而是一个圆锥状的光锥结构，全黑和全白汇聚为中心轴，不同色调沿圆周分布。这也是HSV更符合人眼感知，但空间结构并非正交的原因。

(3) 色域与人眼可见范围

介绍完两种常见的颜色体系，大家通常会关心：人眼可见的光线范围有多大？屏幕、打印机能否完整呈现所有可见颜色？

答案是，设备的显色范围与人眼的可见范围有重叠，但并不完全一致。

人眼的可见光色域整体是一个马蹄形的区域，其中只有30%~60%的区域可以被显示器呈现；相机底片、打印机也只能覆盖其中的部分区域，它们都是可见光色域的子集。

人眼的感知范围远超现有硬件设备，这也是显示技术持续发展的方向。

一个有趣的现象是，早期相机底片的色域要大于显示器和打印机，这也是很多人偏爱胶片摄影的原因之一：底片的成像色域更广，且是连续的、没有明显像素感，视觉感受更舒适。这也是颜色与显示技术协同发展中的一个典型案例。

(4) 伽马校正与颜色位深

另外一个重要的概念是伽马校正。

人眼对亮度的敏感程度是非线性的：白天多开一盏台灯，人眼几乎感知不到差异；但夜晚门缝透出的微光却很容易被察觉。也就是说，人眼在暗部的亮度分辨率更高，在亮部的分辨率更低，均匀的亮度分布对人眼而言信息效率并不高。

因此我们会对图像的灰度值做非线性的伽马映射，将人眼敏感的暗部区域相对拉长，亮部区域相对压缩，让人眼可以更好地分辨图像中的细节。伽马校正是大多数图像软件都会自动完成的基础操作。

硬件领域一直在追求更高的颜色位深。图像最终以离散形式存储和显示，每个颜色通道的比特数（比如8bit、32bit、64bit）决定了颜色可区分的层级，也定义了低通道、标准通道和高通道的颜色显示。

很多时候，人眼感知到的是高动态范围的颜色，但需要在低通道的设备上显示。因此衍生出一系列类似伽马校正的技术，通过非线性映射，在有限的通道范围内尽可能模拟高通道的视觉感受：在高亮区域降低明度以保留对比度，在灰暗区域提升亮度以增强细节，同时保证整体变化连续均匀。这是多通道颜色显示中常见的软件技术。

关于颜色的补充就到这里，我们回到显示技术的讲解。前面我们已经介绍了二维显示，以及彩色背投式阴极射线显示器。

2.4 LCD液晶显示器

人们对显示硬件的追求始终是更便携、更清晰。液晶显示器（LCD）的出现是显示技术的一次重大飞跃，它大幅降低了设备厚度，同时显著提升了视觉分辨率。

LCD的基本结构从底层到上层依次为：背光层、两层偏振片、中间的液晶层，以及彩色滤光片。

- **背光层**：目前绝大多数采用LED作为光源，提供均匀的白光，类似在屏幕后方放置一盏台灯。LCD的主要能耗来自背光层，整体功耗相对较低。统一背光的设计省去了CRT中厚重的电子枪与偏转结构，是屏幕厚度大幅降低的核心原因。
- **偏振片与液晶层**：第一层偏振片控制入射光的偏振方向；液晶层通过改变液晶分子的排列，调节穿过光线的偏振方向。如果液晶层调整后的偏振方向与第二层偏振片完全一致，光线全部通过，对应亮点；如果方向完全垂直，光线完全无法通过，对应暗点。液晶层体积小、响应灵敏，可以高效控制每个像素的出射亮度。

- **彩色滤光片**：在偏振片之后加入红、绿、蓝三色滤光片，使不同像素透出对应颜色的光，最终进入人眼形成彩色画面。

LCD技术发展至今已经非常成熟，它的主要优势是功耗较低；劣势在于背光层常亮，显示纯黑画面时无法完全关闭背光，会存在漏光与余晖，因此屏幕对比度存在上限。

2.5 OLED有机发光显示器

LCD之后的下一代主流显示技术是OLED（有机发光二极管），它与微缩电路技术的发展紧密相关。

LCD的思路是统一发光、通过液晶控制偏振；OLED的思路则是在屏幕上排布大量独立的发光单元，每个单元独立控制颜色与亮度，省去了液晶、偏振控制等复杂结构。显示白色时，发光单元全力发光；显示黑色时，发光单元完全关闭，因此可以实现极高的对比度，视觉效果更纯净。

OLED的性能高度依赖硬件的微缩工艺：发光单元的尺寸决定了屏幕的分辨率。除此之外，它对硬件工艺还有很高要求：

1. 所有发光单元的质量需要高度一致，避免个别单元提前老化，出现坏点影响整体显示效果；
2. 需要保证相邻区域的发光单元使用均匀，减少老化不均的问题。

目前OLED多用于中高端便携设备，电视等大功率大屏仍以LCD为主。OLED的功耗整体高于LCD，且高亮度下发光单元的寿命衰减更快，因此OLED屏幕的峰值亮度通常低于LCD。但在显示质量上，OLED的观感普遍更优。

2.6 色域标准

接下来我们介绍与颜色显示相关的行业标准。

最早的相关标准由美国国家电视标准委员会制定，以高端高动态范围（HDR）显示设备为基准，定义了100%色域的标准，早期只有少数设备能够达到，苹果是较早采用该高标准的厂商。

面向更广泛的个人电脑与打印设备，微软、惠普等厂商制定了sRGB标准，色域覆盖约为基准的72%，对应CIE色度图中更深的蓝色三角形。Windows系统、主流消费级显示器与打印机均采用sRGB色域，是目前最通用的显示标准。

除了显示与打印需求，设计领域对色域有更高的要求。艺术家需要更广的色域进行创作，作品最终可能用于海报打印、3D打印等场景。为此Adobe提出了Adobe RGB标准，对应图中红色区域，色域范围更大。

目前主流的显示与打印硬件还无法完全覆盖Adobe RGB，它更多是设计领域的概念标准，同时也是硬件与打印技术未来的发展方向。

大家未来如果有打印海报等需求，对方通常会询问颜色设置。如果使用了Adobe RGB色域，需要选择转换为标准sRGB还是高动态范围色域，标准色域打印成本更低，高色域打印成本更高。打印时会在目标色域内选取最接近的颜色，还原设计效果。

2.7 分辨率的发展

回到显示设备本身，一个非常直观的趋势是分辨率逐年提升。早年1080P已经属于高分辨率，后来逐步出现4K需求，现在游戏设备普遍追求4K，更高的还有8K。

分辨率的提升对显卡的存储、渲染与刷新能力都提出了更高要求，这部分我们在后续图像章节会详细讲解。超高分辨率不仅用于个人电脑，也广泛应用于大型展板、微缩模型展示等场景，提供更精细的视觉效果。

2.8 立体显示技术展望

以上是平面显示的核心内容，作为后续图像知识的基础铺垫。接下来我们展望立体显示技术，这部分在人机交互章节会有更详细的讲解，这里仅作为显示章节的补充。

(1) 双目视觉原理

人类天生的立体视觉主要来源于双目视觉：两只眼睛存在视角差，看到的物体在遮挡关系、相对位置上存在细微差异。人类经过长期进化，已经可以根据这种差异判断深度：近处物体偏移小，远处物体偏移大，最终从两幅平面图像中恢复出深度信息。这是所有立体显示技术的生理基础。

(2) 偏振3D与头显技术

最早的立体显示技术是偏振式3D眼镜：屏幕发出两组偏振方向不同的光，眼镜上的偏振片分别筛选左右眼对应的画面，大脑自动将两幅画面合成为3D视觉。这是后续所有立体显示的基础原理。

在此基础上，结合头部追踪设备可以实现更沉浸的体验：通过追踪头部位置定位双眼位置，分别计算左右眼应该看到的画面，再由显示设备向对应方向投射光线。这就是裸眼3D与人眼追踪的基本设计。

纯裸眼3D设备只能针对单个人进行投射；如果佩戴专用眼镜，则可以支持多人使用。无论是戴眼镜还是裸眼方案，核心都是结合人眼位置进行定位与画面计算。

(3) AR/VR/MR

我们可以看一下AR/VR的效果演示。图中的画面是合成效果：真实场景中人物站在绿幕中，我们还原了他在头显中看到的内容。VR头显向双眼分别投射对应的画面，营造立体感，结合手柄可以实现空间拾取等交互，是可视交互在虚拟现实方向的重要发展。

- **虚拟现实 (VR)**：呈现完全虚拟的环境，用户看到的所有内容都由计算机生成。
- **增强现实 (AR)**：用户可以直接看到真实环境，设备会在真实场景中叠加虚拟人物或信息，实现真实与虚拟的共存。AR技术在游戏等领域已经有成熟应用。

关于AR、VR以及混合现实(MR)的更多内容，我们在虚拟现实章节会详细介绍。

(4) 裸眼3D与全息显示

除了近眼显示，裸眼3D显示器也是大家很感兴趣的方向。

裸眼3D的核心思路是：在LCD屏幕前增加一层视差屏障，控制光线的出射方向，让不同方向的光线分别进入左右眼。这样一个像素可以拆分为左右眼两个子像素，分别显示对应画面，无需佩戴眼镜即可实现3D效果。

这种方案的代价是屏幕有效分辨率会折半，同时需要底层高分辨率、高亮度的屏幕，以及精准的视差屏障和人眼追踪技术，目前仍在持续研发中。

除了基于屏幕的立体显示，还有更接近真实立体的显示技术：

- **旋转式立体显示**：发光设备高速旋转，依靠视觉暂留在空间中形成持续发光的立体影像，目前仅能在小范围内实现。
- **全息显示**：在立体空间中照射强光，通过全息底片的干涉与衍射效应，在空间中呈现真实的立体光点。全息显示的原理来自物理光学：拍摄时，一束激光照射物体，另一束同相位的参考光直接打到底片上，两束光干涉后，底片记录下物体光波的相位信息；再现时，只用参考光照射全息底片，衍射出的光波与原物体的反射光完全一致，人眼看到的效果就和真实物体相同。

早期全息显示使用物理底片，现在已经发展出数字全息摄影与显示技术，但受限于硬件，目前的显示区域仍然很小。

还有一些更前沿的多模态显示技术，比如触觉显示（通过特定分辨率的纹理让人感知触觉图案）、嗅觉显示（控制物质挥发成分产生不同嗅觉）、体感交互等，这些都属于显示方向的前沿研究，技术尚未完全成熟。

课间休息一下，下节课我们开始讲解图像。

三、色彩在可视化中的应用

我们先补充色彩在可视计算领域的应用案例。

色彩是人眼感知信息的重要渠道，因此也是数据可视化的核心手段。举个例子：统计不同地区的饮料消费偏好，如果只用表格展示数据，很难直观体现地域分布的倾向性。因此可视化中最常用的方法是结合地图，用不同颜色对应不同数据，直观展示区域特征。

但简单的颜色填充也存在问题：一是美观度不足，二是数据重叠区域的信息表达不清晰。

一种常见的改进思路是，根据数据成分对整个区域进行颜色渐变绘制：比如可乐消费高的区域偏蓝，气泡水消费高的区域偏黄，两者混合的区域呈现橙色。这种方式比点状标注更美观，信息也更均匀。

但这种方法依然存在认知误区：如果图例中只有三种颜色，混合出的中间色很容易被观众解读为第四种、第五种独立语义，造成信息误解。这说明不能简单地将颜色直接混合，用户会自然地将混合色识别为新的类别。

可视化领域借鉴了真实世界的颜料混合现象来解决这个问题：

水彩颜料如果提前在调色盘里混合均匀，涂抹后会呈现第三种颜色；但如果先后在画布上涂抹黄、红、蓝三种颜色，颜料无法完全溶解混合，最终画面会保留三种颜色的斑驳痕迹，同时存在局部混合。观众会自然地将其理解为三种颜色的叠加，而不是一种新的颜色。

油画也有类似的特性：颜料干燥后覆盖性强，未干时则可以互相融合。通过控制颜料的融合与稀释程度，可以控制画面呈现“融合”还是“独立叠加”的视觉效果。

基于这个思路，可视化设计衍生出了**噪声叠加混合**的方法：纯色区域使用对应颜色，混合区域则给不同颜色分别叠加随机噪声，让三种颜色以斑驳的形式叠加。最终画面中既有均匀混合的区域，也有保留原色的区域，更接近真实颜料的混合效果，也能更清晰地表达“多语义共存”的含义，而不会被误读为新的类别。

我们可以把这个技术应用到地图可视化中：比如分别用两种颜色可视化致死蚊虫分布区域和活火山活动区域。如果直接颜色混合，中间的黄色会带来语义歧义；如果采用噪声叠加的方式，斑驳的混合区域会被直观理解为两种现象的重叠区域，能更好地展示多维度数据的地理分布。饮料消费的可视化也可以用同样的方法扩展，清晰呈现不同偏好的交界与混合区域。

这是一种更符合人类视觉认知、能准确表达混合语义而非新增语义的可视化方法。

关于颜色的内容就到这里，接下来我们正式讲解图像。

四、图像的定义与表达

课件都已经发布在教学网站上。图像是和显示结合非常紧密的方向，大家对图像都很熟悉，我们先给出它的正式定义。

4.1 图像的数学定义：连续与离散

图像可以定义为二维空间上的**四元组**：

- 前两个元素是像素的空间坐标 (x, y) ，对应二维位置；
- 第三个元素是波长 λ ，对应该位置光的波长与浓度，也就是颜色，可以用RGB、HSV等颜色空间表示；
- 第四个元素是时间 t ，对应曝光时长，会影响画面的亮度。

注：这个四元组里的 t 是图像的**内在属性**，描述的是“这张图是用多长时间曝光出来的”。 $\text{亮度} = \text{光的强度} \times \text{曝光时间}$

这个定义具有普适性：它既适用于电脑中的JPEG、PNG等数字图像，也适用于胶片底片——底片上每个位置的曝光程度、色彩浓度，都可以用这个四元组描述。

这里有一个非常重要的认知：虽然我们日常接触的数字图像是固定分辨率的离散表达，但图像从数学本质上是连续的。胶片底片的曝光是连续能量作用于化学物质的过程，每个点的曝光程度都是连续变化的，因此图像本质上是一个连续的四元函数。

只是在数字计算机中，我们无法保留连续的本质，最终会以离散的方式存储和显示。

这个“连续本质、离散呈现”的趋势贯穿整个计算机图形学领域：真实世界中的物体，无论是图像、立方体、球体还是流体，本质上都是连续的，都可以用连续函数拟合；但最终都要以离散的方式显示在屏幕上。

计算机图形学的核心内核之一，就是建立连续世界到离散空间的映射，通过离散空间的计算解决连续空间的问题——比如渲染解决光照能量分布的问题，仿真解决连续物理场在离散空间的呈现问题，本质上都是在处理两个世界的衔接与转换。

4.2 矢量图与光栅图

将连续图像离散化主要有两种思路：

（1）矢量图

第一种是将图像离散为几何原语，比如点、线段、曲线等。

矢量图的构成是离散的几何元素，但在显示时可以无限缩放，不会失真。它的优势是可以适配任意分辨率，非常适合图标、设计曲线、工程图纸等场景。从硬件角度看，矢量图和早期的矢量显示器、示波器天然匹配，适合精确的曲线展示。

它的不足是表达复杂画面的能力有限，对复杂图像的编辑效率较低。

矢量图广泛存在于我们的生活中，各类系统图标基本都采用矢量格式，目前最通用的格式是SVG（Scalable Vector Graphics）。

SVG文件内部是语义化的元素定义，比如通过polygon标签定义多边形的坐标、填充色和边框，就可以绘制出一个红色填充、黑色描边的四边形；多个四边形组合就可以构成立方体的二维投影。

对于规则图形，SVG可以用极低的信息量实现高质量表达。大家可以在在线编辑器中尝试SVG的编写。

（2）光栅图（位图）

第二种也是我们最熟悉的方式，就是光栅化。

“光栅化”这个名字就来自光栅化显示器：电子束逐行扫描屏幕，逐个点亮像素，图像也就对应地存储为像素的二维数组，每个像素记录对应的颜色。这是现在最主流的图像表达方式。

光栅图的优势是颜色控制简单，适配显示、摄影等绝大多数场景，随着硬件的普及成为了行业主流，这也再次体现了软硬件协同发展的规律。

它的不足是信息量与分辨率直接绑定，缩放时会出现锯齿、模糊等问题；后续我们会讲解反走样技术，就是为了在有限分辨率下尽可能优化显示效果。此外光栅图需要存储每个像素的颜色信息，存储密度远高于矢量图。

(3) 两者的对比

矢量图的核心是连续的几何定义，因此可以根据屏幕分辨率动态计算每个像素的颜色，适配任意分辨率；而位图的分辨率是固定的，放大时会出现明显的锯齿，缩小时也会产生信息损失。矢量图诞生于矢量显示器时代，光栅图则诞生于光栅扫描显示器。如今LCD和OLED成为主流，狭义上的“图像”通常就是指光栅化的像素二维数组。

五、帧缓冲区与GPU基础

5.1 GPU的诞生与图形管线

硬件协同的规律不仅体现在显示硬件定义了数字内容的表达，也体现在计算单元定义了数字内容的计算方式。

当图像以像素二维矩阵的形式存在后，大量图像处理操作都是对每个像素执行相同的运算，比如统一调整亮度、伽马校正等。这类操作高度并行，而CPU的单线程架构并不适合这种简单重复的并行任务，因此GPU（图形处理器）应运而生。

GPU是专门为简单并行运算设计的硬件。它内部包含专门的存储单元，早期完全用于存储下一帧要显示的像素数据；同时集成了大量并行计算线程，专门处理像素级的统一操作。

为了高效完成图形计算，GPU设计了专门的图形管线，可以高速完成下一帧画面的计算与调整，再配合屏幕刷新率，将数据从帧缓冲区刷新到屏幕上，完成显示输出。

后续渲染章节我们会详细讲解**GPU如何从简单的显示辅助单元，逐步发展出完整的图形学功能**，这也是可视计算课程的核心脉络之一。

5.2 帧缓冲区（Frame Buffer）

图像通常存储在帧缓冲区中。下一时刻屏幕要显示的内容，会以二维图像的形式保存在帧缓冲区内，每个像素对应颜色与亮度信息。

显卡上的这块专属存储区域就是帧缓冲区，它会按照屏幕的刷新率（比如60Hz）将数据刷新到屏幕上，这是硬件底层的核心工作。

我们常说的显示器“位深”，指的就是帧缓冲区的比特长度。

- 仅显示黑白内容的简单设备，1bit就足够；
- 基础彩色设备至少需要8bit；
- 主流个人电脑通常是24位或32位；
- 高端专业设备可以达到96位。

从图像表达的角度，帧缓冲区的每个像素可以包含多个通道，对应不同的信息，这是硬件与图像标准协同设计的结果。

最常见的是24位真彩色：将24位拆分为三个8位通道，分别对应RGB三原色，每个通道的取值范围是0~255，总共可以表示约1677万种颜色。这个规格在显示效果和成本之间取得了很好的平衡，因此被称为“真彩色”。

低端设备也会采用每个通道5bit的配置，以牺牲颜色数量换取更低的成本。

5.3 图像存储格式

直接存储全色彩的像素矩阵会占用大量空间，因此衍生出多种图像压缩与存储格式。

(1) 调色板格式 (GIF)

如果一张图片使用的颜色数量远少于1677万，可以将所有用到的颜色存入一个颜色表 (Color Map)，每个像素只存储颜色表的索引，而不是完整的RGB值，从而实现压缩。

GIF格式就是典型的调色板模式，通常使用256色调色板，每个像素仅需8bit。GIF支持多帧动画，连续帧的颜色表通常高度连续，因此可以高效压缩。

调色板的大小可以配置，位数越高颜色还原越好，但压缩率也会相应降低。纯黑白图像甚至可以只用1bit存储，空间效率极高。

(2) 其他常见格式

- **BMP**：无压缩的位图格式，直接存储像素原始数据，体积最大。
- **JPEG**：通用有损压缩标准，压缩比高，广泛用于照片存储，不同质量等级对应不同的精度损失。
- **TIFF**：支持多层图像，常用于摄影、印刷等专业场景，不同图层可以使用不同的位深。
- **PPM**：简单的图像格式，同样支持多层表达。
- **EPS**：常用于字体、印刷领域，文件体积较大，打印兼容性好。
- **PNG**：无损压缩格式，体积比BMP小，压缩率低于JPEG，支持透明通道，是目前互联网的主流格式之一。

JPEG和BMP不支持透明通道，默认以黑色作为背景；PNG支持透明通道，是设计与网页开发的常用格式。

5.4 Alpha通道与图像合成

帧缓冲区（帧缓冲区是显卡上一块专门的内存，用来存放「准备输出到屏幕上那一张完整画面的全部像素数据」）的位深通常会高于24位，额外的位数用于存储更多信息，其中最常见的就是Alpha（透明度）通道。

帧缓冲区的绘制是分层进行的，后绘制的内容会覆盖之前的内容，Alpha通道用于控制新图层的透明度，决定背景可以透出多少。最终画面混合完成后，未被不透明像素覆盖的区域会填充背景色，再输出到屏幕。

Alpha通道的取值范围是0~1：

- 1.0表示完全不透明；
- 0.0表示完全透明；
- 中间值对应半透明效果，非常适合表达阴影、光晕、光标、带透明效果的Logo等。

最基础的线性混合公式为：

$$\text{最终颜色} = \alpha \times \text{前景色} + (1 - \alpha) \times \text{背景色}$$

实际设计中，Alpha通道通常是0到1之间的连续过渡，而不是非0即1的硬边缘。

Alpha通道广泛应用于影视制作：前景在绿幕中拍摄，后期将绿色区域的Alpha值设为0，就可以叠加任意背景，这就是绿幕抠图的核心原理。CG虚拟角色与实景的合成，也依赖多层Alpha通道的叠加计算。

5.5 深度缓冲 (Z-Buffer)

除了Alpha通道，帧缓冲区中还会存储深度信息，也就是Z-Buffer（深度缓冲区）。

3D场景最终要呈现在2D屏幕上，每个像素对应的空间点到相机的距离，就是深度值。

绘制多个物体时，通过比较深度值可以判断哪个物体在前方、哪个在后方，决定该像素最终显示哪个物体的颜色，以及透明混合的顺序。这些逻辑现在都已经固化在GPU硬件中。

深度信息通常不用8bit存储，至少使用16bit，很多时候采用指数形式存储。原因是人眼对近处的深度变化非常敏感，遮挡关系是很强的视觉线索，需要高精度；而远处的深度精度要求较低。指数存储可以在近处提供更高的精度，远处精度较低，符合人眼的感知特性，同时也会占用更多的位深。

感觉很多时候就是结合人的心理来做的啊...也有道理，毕竟“显示”本身就是以人作为最后的消费对象来发展的

5.6 双缓冲区

随着帧率提升，硬件厂商普遍采用双缓冲区机制。

双缓冲区同时维护两张图像：一张是当前正在屏幕上显示的画面，另一张是正在后台计算修改的画面。修改完成后，两个缓冲区瞬间交换，用户看到的始终是完整的画面，避免了画面撕裂，也隐藏了渲染过程。

双缓冲区会让帧缓冲区的总容量翻倍，比如原本48位的缓冲区会变为96位。大家选购显示器时，可以关注帧缓冲区的位深参数。

六、图像处理概述与课程预告

当图像以离散二维矩阵的形式存在时，连续到离散的映射会产生各种问题，因此需要图像处理技术来弥补。

图像处理可以按照处理范围分为三类：

1. **逐点处理 (Point Operation)**：对每个像素独立进行运算，像素之间互不影响。比如图像拷贝、反色、伽马校正、亮度调整、曲线调色等。这类操作最简单，也最适合GPU并行计算。
2. **区域处理 (Filtering, 滤波)**：对像素的邻域进行运算，提取图像的连续特征，比如去噪、边缘检测、模糊等。滤波和数字信号处理、信号传输理论联系紧密，本质是在离散表达上运算离散算子，提取连续意义上的信息。
3. **全局处理**：对整幅图像进行操作，比如图像变形、压缩、语义分割、背景剔除，以及更高层的图像理解、视频处理等。

今天由于时间限制，我们不会深入讲解区域处理。后续我们会按照从局域到全局的顺序，依次讲解：

- 抖动技术：用更低的颜色位深模拟更丰富的颜色观感；
- 图像滤波；
- 图像补全、背景匹配；
- 直线生成、三角形光栅化、反走样、曲线绘制等内容。

这些都是课程基础部分的核心重点，也是第一次考试的主要考察内容。

课程预告

明天我们会在课程网站上发布实验1 (Lab 1)，对应前面的基础内容，以图像处理为主。

实验内容包括图像抖动、滤波、边界处理、图像融合等，完成代码填空后就可以看到对应的处理效果。大家可以结合实验复习课程内容，掌握具体算法。

考试主要考察算法与计算，会提供相关公式，不需要死记硬背。考试说明会发布在课程网站上。

今天的内容就到这里，明天再见，谢谢大家。

LESSON 2 0915早十

图像处理：抖动、量化、滤波、边缘提取、补全与抠图

一、图像处理引入

今天讲图像处理部分，承接之前的图像表达。图像处理主要针对离散的、光栅化表示的图像。我们走一个从局部到全局的过程：一开始是局部计算，之后逐渐涉及全局约束、需要统一求解的运算。这是比较重要的算法内容，也是考试中比较重要的一部分。

二、抖动与量化

1. 背景

图像处理最前面的问题是抖动和量化。这里的量化，指的是把图像压缩成位数更低的表达。这是一个有实际应用需求的问题。

最初，人们把图像作为一种表达分离出来之后，可以存一个比较大的文件，表达比较多比特的图片，比如一张 8 比特灰度图（注：每个像素用一个 8 位二进制数来表示亮度，只有明暗信息，没有颜色信息。能表示 $2^8=256$ 个不同的灰度等级）。但图像最终还要在各种显示设备上显示。有时设备只有 1 bit，只能显示黑白两色；有时输入是 24 位真彩色 RGB 图像，但显示设备只能显示 8 比特灰度。因此，我们希望有一些压缩方法，使图像在这些设备上仍有较好的展示效果。这就是抖动和量化的背景。

2. 直接二值化的问题

如果把一张 8 比特灰度图显示成黑白图像，最简单的方法是阈值截断：灰度大于等于 0.5，或大于等于 128，就置为白色，即 255 或 1.0；小于阈值就置为 0。这样会得到一张黑白图。

对每个像素来说，这种做法的误差已经最小。但人们发现，这种视觉成像有很多不令人满意之处，于是提出了抖动。

误差不仅有大小的区别，还有频率的区别。将原图和二值化结果做差，得到二维误差图。对误差图做傅里叶分解，就转换到频率空间。我们关心不同频率 k 上误差的能量或振幅。以频率为横轴、能量为纵轴可视化，会发现：直接二值化时，每个像素误差最小，但误差全部处于最低频率，几乎没有周期性变化。

注：1. 误差图：对每个像素，误差就是：

$$e(x, y) = I_{\text{原图}}(x, y) - I_{\text{二值化}}(x, y)$$

例如：原像素是 200，二值化后是 255，误差是 $200 - 255 = -55$ ；

2. 频率代表什么：表示信号变化的快慢。在图像里，低频意味着图像在很大范围内缓慢变化，比如大块平滑过渡或纯色；高频如边缘、纹理、噪点。

3. 为什么误差都在最低频？直接二值化后，误差图往往是这样：

- 一大片浅灰色区域，所有像素都变成白色，误差都是差不多的负值；
- 一大片深灰色区域，所有像素都变成黑色，误差都是差不多的正值。

所以误差在空间上变化很慢，几乎没有频繁跳动。

对这样的误差图做傅里叶变换，能量就集中在低频。
高频部分几乎为零。

这就是“误差全部处于最低频率，几乎没有周期性变化”。

3. 抖动的思想

抖动做了一个有趣的事情：先给图像随机加入人为噪声，再做二值化。直觉上，加噪声会丢失信息，再做二值化又进一步丢失信息。但从视觉上看，很多人会觉得这张图传递了更多原始图像的信息。

同样可以计算误差并做频率分解。直接二值化的逐像素 L2 loss 可能更小，但抖动后的误差分布在低频和高频都有。人类视觉对低频误差非常敏感：大片区域系统性偏低或偏高，会被认为两张图差异很大，比如对比度不同、细节特征不同。但高频误差，例如局部噪点，人眼由于进化原因反而不敏感。

因此，虽然每个像素本身的选择不一定最优，但均匀噪声在局部区域中减少了低频偏差。所以，

抖动可以定义为：

把结构化的、容易被人眼察觉的低频量化误差，转换成频谱更加分散的噪声。

三、白噪声与蓝噪声

随机噪声称为白噪声。对噪声做空间到频率的傅里叶变换，会发现白噪声完全随机，因此频率分布比较满、比较连续。

另一种噪声只包含高频信息，称为蓝噪声。给原图加蓝噪声后，低频区域更接近原图，误差被累积到高频。人眼对高频不敏感，所以图像主体、黑 T 恤、眼镜等低频特征仍然可识别。

注：蓝噪声常常表现为，点分布比较均匀、不会出现大片空白也不会很多点挤在一起。

对比误差能量，蓝噪声的总能量甚至可能比白噪声更高，但误差都分布在高频，低频误差被抑制，因此视觉感受更好。

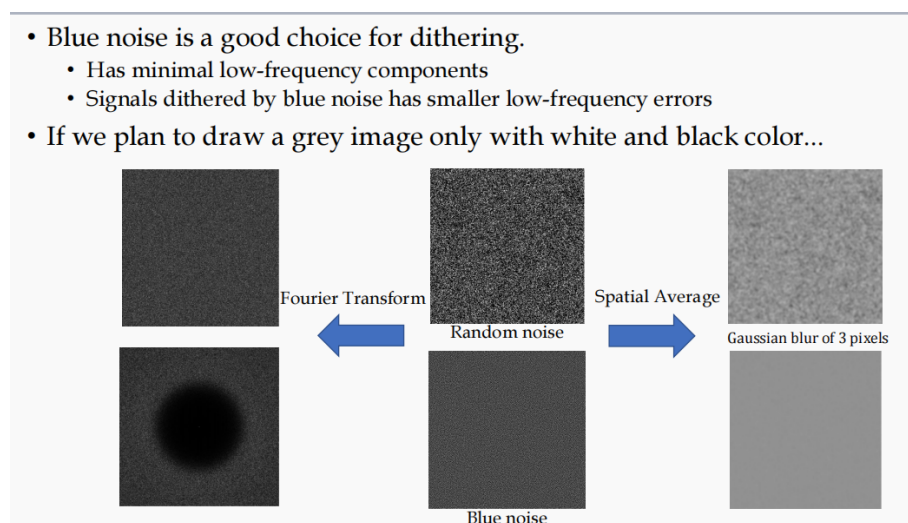
白噪声在每个频率都有信息；蓝噪声在低频范围内噪声较小，主要信息在高频循环往复。日常生活中，白光频谱齐全，是复合光；蓝光以高频为主。这里借波动光学的角度理解图像信息处理。

图像方向与信号处理、数据处理密切相关。很多图像处理会转换到频域，分析高频细节和低频结构。

四、滤波与模糊

噪声可以作为图像信息源，再应用图像滤波。常见滤波包括模糊处理。模糊又叫**低通滤波**：它作用在高频信号上，把高频信号模糊掉，只让低频信号通过。高斯模糊或其他模糊常被称为低通滤波。

模糊是低通滤波，因为模糊会把每个像素和周围像素平均一下，快速跳变的部分被抹平了，只剩下缓慢变化的部分。所以说蓝噪声很适合做dithering。



如果做一个 3 像素高斯滤波（高斯滤波是一种用高斯函数作为权重的模糊滤波，可以理解为：对每个像素，用周围像素的加权平均来替换它。离中心越近的像素权重越大，离得越远的权重越小。权重按高斯分布衰减。），对于均匀噪声，仍会留下更低频的噪声底色；对于更高频的噪声，会得到接近均匀的灰度图，因为高频细节都被滤掉。这里也体现了可视计算的优势：想分析什么，就把它可视化，可视化结果通常很有说服力。

五、有序抖动与空间换颜色

加噪声是一种滤波思路，但早期受限于 GPU 上的噪声不容易获得、并行性不好等问题。若要把抖动做成硬件中的可执行程序，噪声可能不友好。解决手段包括预存一张噪声图片，但可能受图像大小影响。总之，加噪声有不足。

另一个研究方向是用更大的空间换取颜色通道上的不足。例如用 3×3 区域表达本来一个像素要表达的信息，从而获得更多灰度阶。全 0 表示完全黑色，全 1 表示完全点亮，这样 3×3 可以传递比 0/1 更多的信息。

这里最核心的是设计有序抖动（有序抖动：按照一个固定的顺序，依次点亮不同数量的像素，以表示中间灰度）。0 和 9 比较明确，但中间要先点亮哪个像素、按什么顺序点亮，需要用空间组织。成熟结果中，一个重要性质是：先前点亮过的像素，后续必须持续保持点亮。同时希望相邻之间连续。因此会设计出 pattern：一开始点亮某个像素，之后逐次点亮邻居，直到 9 个全部点亮。

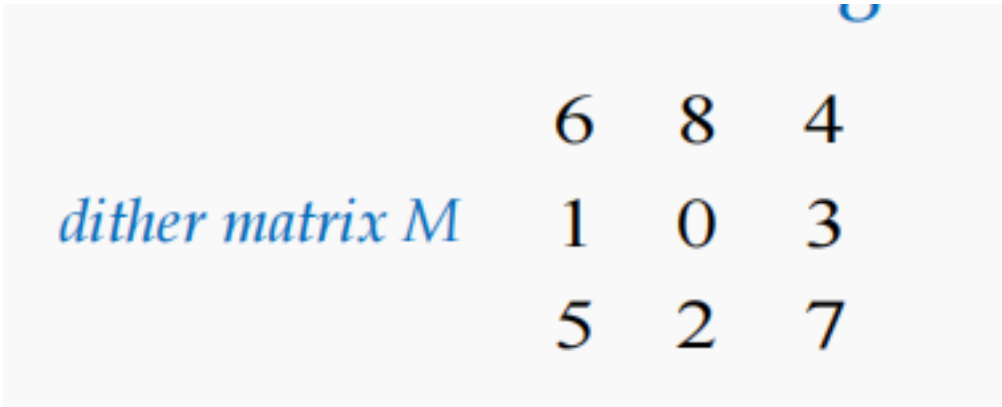
为什么一定要连续？如果随机点亮，从信息传递上似乎没有不同。但抖动不仅用于当前图像，也可用于随时间变化的图像。若输入是视频，可以对时间做同样抖动处理。在黑白设备上做动图展示时，抖动可以让中心较亮、周围以灰度逐渐过渡，虽然是二值图像，但能看到明暗渐变。

如果按顺序设计 pattern，当某个像素值从 3 变到 4、5，再回到 4、3，连续变化时，整个 pattern 也保持连续变化。在动图中，人不会注意到信息抖动，而会关注图像渐变本身。这是有序变化的一个理由。

在静态空间中，人也不对不规则突变更敏感。若 1 显示一个点，2 突然显示对角两个点，相邻区域从 1 变到 2 时，不仅亮度变化，还有结构变化。拼在一起后，容易注意到本来不应有的结构错觉，形成图形学中的 artifact。连续设计则让人眼认为信号有结构性持续变化，更多识别渐变过程，而非结构化改变。

使用 3×3 空间换颜色的方法，很多时候是因为硬件有并行需求。查 pattern 表格可行，但算法上希望更优化。于是可以设计抖动矩阵，标注依次点亮像素的顺序。例如最首先点亮中间，然后左边，然后底下。只要表示矩阵，就记录了 pattern 变化轨迹，不必存储所有 pattern。

(pattern 表格是指：为每个灰度级预先存一张黑白图案，用的时候直接查表。抖动矩阵是指：不存所有图案，只存一个“顺序矩阵”，用的时候通过比较动态生成图案。)



	6	8	4
<i>dither matrix M</i>	1	0	3
	5	2	7

处理图像时，把待处理像素归一化到 0~9。若当前像素值为 4，则所有小于 4 的都应已点亮，大于等于 4 的尚未点亮。这样可以并行、快速地决定元素是否点亮。

抖动常存储抖动矩阵。但这增加了分辨率。如果不想让硬件组装更多屏幕，能否用同样分辨率、有序抖动、适合并行且不需要提前搞噪声？答案可以。

方法是图像切成 3×3 格子。把每个 3×3 图像块与抖动矩阵比较。两边都归一化，例如抖动矩阵从 0~8 归一化到区间。若当前像素块的值大于抖动矩阵对应值，就置为点亮；小于则熄灭。这样把连续空间转换为固定 pattern，同时保持连续性。

这样不增加显示分辨率，也能做有序抖动。在图像上做有序抖动，即使不人为添加噪声，抖动本身有一定非规则性，人眼能忽视高频规则信息，效果较好。

对比白噪声抖动、蓝噪声抖动、规则有序抖动，有序抖动感受上介于前两者之间。蓝噪声早期若没有预存，生成较复杂。从信号处理角度，白噪声是全频谱噪声，蓝噪声是高频噪声、低频没有。规则有序 pattern 做频谱分析，会发现它存在于某几个特殊频率上，相对较高频，因此效果介于二者之间。

六、Floyd-Steinberg 抖动

历史上还有一些抖动，现在用得少。例如与光栅化硬件协同发展的 Floyd-Steinberg 抖动算法。当时并行性不好，没有 GPU，只有扫描线逐像素处理流程。人们希望不关心全局，逐一确定每个图像信息。

Floyd-Steinberg 抖动的基本想法：对第一个像素，直接判断大于 0.5 还是小于 0.5，确定 0 或 1 的二值。确定后存在误差，把误差算下来，传播到后面像素，让它们分担并校正误差。处理到 T 型位置时，误差只能传给后面尚未处理的像素。空间设计上，较近的像素承担多一些。误差分配后，若某像素偏暗，其他像素可能被点亮。再处理这些像素时，做简单二值化，就能把某区域误差均匀分散到全空间，使误差从低频走向高频。

现在 GPU 并行很好，这个算法用得不多，但当时对扫描线设备能起到接近蓝噪声的效果。

抖动可以扩展到 RGB 颜色空间。无论使用抖动矩阵还是 Floyd-Steinberg，都可以对红、绿、蓝分别抖动。这适合把 24 位彩色图应用到每个通道只能存较低颜色的设备。直接二值化彩色图会导致色调明显变化，因此抖动在资源有限时非常有用。

图形学中，蓝噪声已经比较容易获取，推荐使用蓝噪声。图形学很多时候需要用离散表达连续世界。虽然图像抖动听起来是历史技术，图像分辨率也足够多，但如何用有限空间更好表达连续世界，是永恒问题。这不仅存在于图像处理，也存在于几何建模中采样点、曲率、微分几何信息。如果采样点不够密，可以给原始信息加蓝噪声，使最终采样信息在频率上更好反映连续空间。渲染中，尤其光线追踪模拟光线在空间中的传播和能量分布，计算复杂度很高，只能模拟有限几束光线，因此需要好的 sampling 技巧。取样函数值时，加白噪声或蓝噪声往往有用。抖动是基础且与任何采样相关，蓝噪声也因此非常有用。

七、图像滤波与卷积 *image filtering*

讲完抖动，它是在局部、和硬件联系紧密的优化。接着讲更常见的图像处理。在 CV 领域，特征提取很重要，与之相关的是 filter。

最简单的 filter 是模糊。模糊往往和频率相关：滤掉某些高频信息，只通过低频信息。模糊核决定哪些高频被滤掉、剩下哪些低频。图像分辨率不高时，例如 64×64 ，分别做 3×3 或 5×5 模糊，高频信息逐渐减少，剩下信息越来越偏低频。

模糊是常见的区域性图像处理手段。最简单的模糊是均值滤波。把 3×3 区域中以某像素为中心的邻居全部加权、相加、求平均，用平均值覆盖本来的像素颜色值，就完成局部均值模糊。

也可以写成矩阵计算形式。模糊核写成 3×3 矩阵，每个数代表对应像素点应乘以的权重。可以扩充到 $N \times N$ ，只需适度归一化，使图像不会过亮或过暗。

从数学上看，filter 定义了一种卷积操作。连续空间中的卷积：

$$(FG)(t) = \int F(\tau)G(t - \tau) d\tau.$$

注：连续空间中的卷积公式

$$(F * G)(t) = \int F(\tau)G(t - \tau) d\tau$$

-
- F : 输入信号，比如一段声音或一张连续图像。
 - G : 卷积核，比如高斯核。
 - t : 当前时刻或当前位置。
 - τ : 积分变量，遍历整个信号。
-

- $G(t - \tau)$: 把核 G 翻转并平移 t 。

这个公式的意思是：输出在 t 处的值，等于输入 F 和翻转平移后的核 G 在整个区间上的乘积积分。

图像是离散的，二维离散卷积：

$$(FG)(x, y) = \sum_{m, n} F(m, n)G(x - m, y - n).$$

连续定义可用无穷大空间和无穷大卷积核。实际使用中，常用有限维度卷积核，认为离当前像素太远的信息不重要，因此远处卷积核 $G=0$ ，只有 3×3 或 5×5 区域有值。手工设计卷积核时，常设计对称或反对称核，认为其具备一定空间性质。

这里我纠结了很久的一点在于，卷积核为什么要先翻转。最后得到的结论是，数学定义有关。翻转后的东西叫“相关”(correlation)

卷积核 $G(t - \tau)$ 可看作把无穷维函数 $G(\tau)$ 镜像成 $G(-\tau)$ ，再向右平移 t ，与图像对齐。二维可视化时，输入为 F ，有限 3×3 核先原地镜像，然后随处理数据移动。卷积时对应项相乘，最终生成输出。输出每个点的值，等于卷积核与它对齐后对应位置数值两两相乘并叠加。

卷积运算可交换：用 A 卷积 H 和用 H 卷积图像 A ，最终效果对称一致。

八、边缘提取、Sobel 与拉普拉斯

卷积可以提取信息，尤其高频信息。先看人为设计的卷积核。

假设输入图像左边全黑，右边全亮，例如 25。应用一个对称但取相反值的核，对每个像素，相当于右边减左边，并按同样比例相减。这其实是有限差分：总是用右边的数减左边的数。如果周围值完全一样，右边减左边没有东西；但在 0 和 25 的交界处，左右邻居出现跳变，右边减左边会准确捕捉跳变，并存储有限差分。到 25 这边后，因为完全一致，继续保持 0。

逐像素做卷积操作，非常适合并行。结果中，所有跟周围邻居发生突然跳变的信息点被保留，其他信息没有。这就是边缘提取：把颜色突变、两个不同区域之间的边提取出来。

Examples: edge filter

$$b[x, y] = \sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} a[u, v]h[x - u, y - v]$$

$a[x, y]$ = input signal
 $b[x, y]$ = output signal
 $h[x, y]$ = 3×3 filter
 x, y take on only integer vals

$$\frac{1}{4} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

h
(0,0) at center

0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25
0	0	0	0	0	25	25	25	25	25

a

0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0
0	0	0	0	0	25	25	0	0	0

b

水平方向有限差分展示竖直边；垂直方向有限差分展示水平边。采用水平方向和垂直方向的有限差分，可以分析图像中竖直和水平分割线。竖直和水平方向都重要，因此建立常用边缘提取卷积方式，逐渐固化形成各种算子，例如 Sobel 算子。

Sobel 算子思路：先用水平方向提取水平方向的边变化，再用垂直方向导数计算垂直方向的边信息。得到两张卷积图后，分别平方、求和、开根号：

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}.$$

1. Sobel 的两个卷积核

Sobel 算子有两个 3×3 核：

水平方向梯度核 G_x ，用来检测**竖直边缘**：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

垂直方向梯度核 G_y ，用来检测**水平边缘**：

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

在图像处理中，通常直接拿这两个核在图像上滑动，做“对应位置相乘再求和”。严格说这叫**相关**，但因为边缘检测只关心梯度大小，符号不重要，所以大家常直接叫它卷积。

2. 准备一个简单的图像块

我们取一个 3×3 的图像块，左边两列暗，右边一列亮，形成一个竖直边缘：

$$I = \begin{bmatrix} 10 & 10 & 50 \\ 10 & 10 & 50 \\ 10 & 10 & 50 \end{bmatrix}$$

我们计算**中心像素** $(1, 1)$ 处的 Sobel 响应。中心像素的值是 10。



5. 计算梯度模长

Sobel 算子的最终输出是梯度模长：

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

代入：

$$|\nabla I| = \sqrt{160^2 + 0^2} = 160$$

结果很大，说明中心像素处存在很强的边缘。

6. 结果解读

- $G_x = 160$ ：水平方向变化剧烈，说明有**竖直边缘**。
- $G_y = 0$ ：垂直方向没有变化，说明没有水平边缘。
- 梯度模长 160：边缘强度。

这正好符合我们的图像：左边暗、右边亮，中间有一条竖直的分界线。

每个像素自身完成操作，最终得到边缘提取函数，保证横向和纵向边信息都被提取。

图像本身是连续信号源，只是以离散方式表达。很多运算符合这个思想：在离散表达上运行离散运算，例如离散卷积，但存在来自连续空间的含义。有限差分对应微分运算：求图像在水平方向的变化率，以及竖直方向的变化率。微分从连续空间离散到图像空间，就得到有限差分计算核。它会带来离散误差，但在离散空间中已经是对连续函数的最好近似。图像空间中将连续函数对应的离散处理方式称为图像离散算子。

Sobel 算子计算的是梯度模长：

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

图像作为连续信号，在空间中变化。有的地方变化快，有的地方变化慢。梯度模长能告诉哪些地方变化快，哪些地方没有额外信息、比较不重要。因此 Sobel 卷积能提供边缘提取效果。

拉普拉斯算子在图形学中非常重要，后面还会多次接触。数学上，拉普拉斯算子计算二阶微分：

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

2. 二维拉普拉斯算子

图像是二维的，有 x 方向和 y 方向。

拉普拉斯算子的连续定义是：

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

也就是： x 方向二阶微分 + y 方向二阶微分。

分别离散化：

• x 方向：

$$f(x+1, y) + f(x-1, y) - 2f(x, y)$$

• y 方向：

$$f(x, y+1) + f(x, y-1) - 2f(x, y)$$

两者相加：

$$\begin{aligned} \nabla^2 f \approx & f(x+1, y) + f(x-1, y) - 2f(x, y) \\ & + f(x, y+1) + f(x, y-1) - 2f(x, y) \end{aligned}$$

合并同类项：

$$\nabla^2 f \approx f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

这就是离散拉普拉斯算子。



离散化时，先用有限差分表达一阶微分，再用一阶微分结果的有限差分表示二阶微分。拉普拉斯是二阶微分的和，因此把对角线相加，得到拉普拉斯。写成卷积矩阵：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

它非常对称，中心为 -4 ，周围为 $+1$ 。用拉普拉斯算子作为二阶微分，体现的是当前数据与周围数据的差异。若都一致，则没有信息；若明显比周围点大或小，则是局部极值点。这也符合二阶微分理解。**拉普拉斯算子也可用于边缘提取。**

均值滤波器只有 3×3 部分有均值。转换到频率空间，它完全存在于低频通道，高频几乎没有。把频率信息卷积到图片上，低频保留，高频被滤掉，因此模糊能对图像起到高频滤波效果。

均值滤波简单，但不能很好保持周围信息，容易把规则区域一块块模糊掉。后来提出高斯滤波。核心思想是：均值滤波转换到频率空间后，频率空间不太连续，低频留下，高频有的留有的不留，不够优雅。高斯滤波希望关注低频信息，高频信息逐级减少。因此用高斯核做内部卷积加权。高斯核转换到频率空间，从高频到低频是比较均匀的过渡。用高斯核处理原始图片，能更好保留低频信息，高频逐渐淡化，更好体现图像低频信息，整体 artifact 更少。因此模糊中更常

用高斯滤波。

课间休息。

九、频率的直观理解

有同学问频率该怎么理解。对二维图像做傅里叶分解，可以理解为：手上有一张 256×256 的图片，用一堆棋盘格叠加起来拟合它。棋盘格有不同频率： 1×1 纯底色， 2×2 黑白， 4×4 ， 6×6 ，甚至 256×256 。到 256×256 时更接近高频、更不规律。每个棋盘格乘以强度，并可能移相位。所有棋盘格叠加，最终累计获得能近似原图的照片。因此，图像这个连续信号在空间中的频率变化，被拆解到各个频率维度上。

图像滤波和特征提取在 CV 和机器学习中很重要。过去很多图像理解的信息提取工作，逐渐从手工算子发展到机器学习、数据驱动。理解某一频率上的信息很有用，因为机器学习不应永远是黑盒。图像处理提供了很多对数据驱动学习的原理理解。例如图像网络中，第一层、第二层卷积核提取边或角，越往后越能提取边和角共存的高层、抽象结构、语义信息，最后识别出猫或狗。这些都与图像 filter 有助于理解机器学习。

历史上，过去认为局部信息可以由黑盒深度学习学好；深度学习未发展时，大家认为更全局的信息，例如图像补全，机器学习做不好。现在生成模型可以直接生成图片，这些问题已被解决。但从局部到全局的思考，有助于理解深度学习做了什么。

十、图像补全与拉普拉斯编辑

图像补全是相对全局的任务。数学上，假设有一张图片，中间被挖掉一块，或有一些不想要的图案被抠掉。我们希望填补这块内容。需求包括：边界从容过渡，边界信息和已知边界条件完全一致；内部填补时，若只是简单抠掉，一般希望不填补任何信息，像平静水面，中间什么信息都没有。

已经学过边缘提取算子和微分算子。如果某区域有微分，人眼会看到差别和内容。因此补全的核心思想是：填完后，该区域在微分上没有任何信息。理想状况是边界契合，整个区域连续变化。用微分算子跑边缘，希望微分意义上没有任何信息，即非常连续的变化，这样图像被完整补好，且没有提供不应提供的信息。

图像补全来源于实际生活。现在手机拍照片，选取路人，让手机补全，通常用 AI 算法，效果往往优于传统算法。但传统算法完全从数学出发，有助于理解这件事。

补全涉及的概念：第一，忠诚于周围邻居，也就是边界必须保持一致，没有更改。其他图像处理中，不想修改的部分也应尽可能保持本来信息。第二，遵循剃刀原则：如果只是补全，没有特殊要补的东西，那么所有东西都不应被增加，增加的信息量尽可能少。

我们希望微分量，例如梯度，越小越好。图像补全公式化：要补全的区域为 F ，给定图像为 F 。希望补全区域在边界 $\partial\Omega$ 上对待补全图像边界一模一样，不出现跳变。在区域 Ω 内，关心梯度模长的平方积分，希望值越小越好：

$$\min_F \iint_{\Omega} \|\nabla F\|^2 dA, \quad F|_{\partial\Omega} = F|_{\partial\Omega}.$$

1. 符号含义

$$\min_F \iint_{\Omega} \|\nabla F\|^2 dA, \quad F|_{\partial\Omega} = F^*|_{\partial\Omega}.$$

- F : 待求的补全图像。它在区域 Ω 内每个像素的值是未知的。
- Ω : 待补全的区域, 也就是被挖掉的那块洞。
- $\partial\Omega$: 区域 Ω 的边界。
- F^* : 已知的原图。边界上的值是已知的。
- ∇F : F 的梯度, 表示图像在 x 方向和 y 方向的变化率:

$$\nabla F = \left(\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y} \right)$$

- $\|\nabla F\|^2$: 梯度模长的平方:

$$\|\nabla F\|^2 = \left(\frac{\partial F}{\partial x} \right)^2 + \left(\frac{\partial F}{\partial y} \right)^2$$

它衡量图像在该点变化有多剧烈。

- $\iint_{\Omega} \|\nabla F\|^2 dA$: 把整个区域 Ω 内所有点的梯度平方加起来, 得到总能量。
- $\min_F$: 寻找一个函数 F , 使这个总能量最小。
- $F|_{\partial\Omega} = F^*|_{\partial\Omega}$: 边界条件。在区域边界上, 补全结果必须和原图边界完全一样, 不能有跳变。

极端情况下, 边界全是常数 1, 则区域用纯色 1 填充, 完成最小化。但边界往往是任意曲线, 上面的值任意。因此需要计算手段, 从数学上解决能量最小化问题。这个问题后来常称为拉普拉斯图像编辑。

图像补全可以很厉害。图像由很大色块组成, 若图像只剩几条边, 每个区域都有完整闭合边界条件, 可以对每个最小闭合区域应用拉普拉斯图像补全: 希望梯度整体没有掉, 边沿完全准确符合, 然后填充。最终能很大程度保留原始图像信息。这说明拉普拉斯图像补全是有意义的、不产生额外信息的操作手段。

若直接思考怎么做, 因为它是能量最小化问题, 可以先随机生成所有点的像素值, 获得一张图像。在图像上计算梯度, 计算边界与这些边的误差。把所有能量作为总能量, 做梯度下降优化。每个像素调整自身值, 使它与周围梯度的信息逐渐减少, 与边界上点的信息逐渐接近。优化很多步后, 可以得到图像补全结果。这是基于梯度下降优化的算法, 可能不高效, 但能完成能量最小化任务。图像补全发展较久, 数学上可以较好求解, 因此后面有更高效算法。

十一、图像克隆、融合与泊松编辑

图像补全更进一步是图像克隆、融合或图像融入。需求可能是: 已有背景, 另一张图有更感兴趣的前景, 想把前景挖出来放到背景中。如果直接把两张图通过不透明度简单叠加, 会出现不融合效果。

图像融合想做的事情与补全类似。希望图像所有边界信息与背景图本来边界信息保持一致, 而内部所有信息与前景信息保持一致。内部保持一致又涉及微分、梯度。计算每个像素梯度, 希望梯度尽可能与前景图像梯度一致。然后做能量最小化:

$$\min_F \iint_{\Omega} \|\nabla F - \nabla G\|^2 dA, \quad F|_{\partial\Omega} = G|_{\partial\Omega}.$$

在边界完全等于已知边界的情况下，最小化内部梯度与前景已知梯度的差。做到最小化，就能完成图像融合。结果中内容比较均匀地过渡，前景细节较好融合到图片中。这是从图像补全扩展到图像克隆。

这个技术现在也可被 AI 替代。AI 可以完成前景和背景融合，甚至效果更好。例如背景图和前景图色差过大，前景搬过来后会出现较大色差、整体低频系统偏差，对语义理解产生影响。后来 AI 有更好的融合手段，但相对黑盒。

图像补全和图像克隆都属于泊松编辑范畴。图像补全称为 Laplacian editing，图像克隆称为 Poisson editing。

数学抽象：任务依然是猜 F 。希望边界上等于已知边界，但此时要最小化的不是全部梯度，不是希望没有任何信息，而是希望信息完全和已知某图像信息一致。通常 G 来自想要的前景，来自另一张图片。提取它的梯度，就是一个合法图像梯度。希望当前梯度尽可能接近目标梯度。函数最小化时，得到的 F 就能起到图像融合效果。

从这个例子看，图像补全，即 Laplacian editing，是 Poisson editing 的特例：要补的图片恰好是均值图片，梯度完全等于 0。

能量最小化问题怎么求解？这部分内容不考，只需记住结论：这是算法要实现的内容。最小化能量随自变量 F 变化，而 F 本身是函数，因此是泛函分析中的能量最小化、变分问题。若存在泛函

$$G[F] = \iint g(F, F_x, F_y, \dots) dA,$$

从变分角度构造欧拉-拉格朗日方程。欧拉-拉格朗日方程等于 0 时，泛函达到最小。图像是二维情况，二维欧拉-拉格朗日方程有相应形式。

把泛函带入，整理后得到：

$$\Delta F = \Delta G.$$

也就是说，优化梯度之差尽可能接近，等价于在区域 Ω 中，填补函数 F 的拉普拉斯，即二阶微分，与目标前景的拉普拉斯完全相等。某个函数的拉普拉斯算子等于固定值，这类方程称为泊松方程。这就是图像处理被称为 Poisson editing 的原因。

后续图形学会多次讲拉普拉斯算子和泊松方程求解。特例是梯度恒等于 0，则拉普拉斯也等于 0。此时泊松方程变为拉普拉斯方程：

$$\Delta F = 0.$$

这解释了图像补全为什么叫 Laplacian editing。

十二、泊松方程的离散求解

即使知道最小化解是 $\Delta F = \Delta G$ ，仍需解方程。在离散意义上，拉普拉斯等于固定值。固定值可以用 image filter 技术从图像算出。因此右边已知。

微分可用有限差分近似。二阶微分要用两次有限差分。对某一像素，拉普拉斯离散形式是上下左右邻居再减去 4 倍自己：

$$F_{i+1,j} + F_{i-1,j} + F_{i,j+1} + F_{i,j-1} - 4F_{i,j} = b_{i,j}.$$

每个点希望拉普拉斯等于固定值，就列出该点与上下左右邻居和自己的方程。有限图像上有很多像素点，每个像素点给一个方程。未知量是所有待填像素信息，方程个数由每个点提供，还有边界信息。

例如某点左邻居和下邻居已知，就把已知值带入，消掉未知量。这类边界条件称为第一类边界条件，即 Dirichlet 边界条件。后面讲物理仿真时也会讲边界条件很重要。

若填补一大片区域，每个点列拉普拉斯方程，组成对称、中心主轴重要的矩阵。每个点有带求量，存在 Dirichlet 边界条件，可把已知量移到等式右边，变成边界条件。整个问题转化为

$$AF = B$$

的矩阵求解。 F 是所有待求像素值， B 是已知边界信息。左侧矩阵绝大多数是四负一形式，具备对称、稀疏等良好性质。有高效求解矩阵的手段，最终可求

$$F = A^{-1}B.$$

习题要做的图像补全，主要内容是构造拉普拉斯矩阵和右边边界项。构造完 $AX = B$ 后，求解 X 可调用现有算法。习题代码提供了解方程方法，用的是 Jacobi 迭代矩阵求解。迭代求解可以随机初始化 F ，计算拉普拉斯算子残余，把残余减过来。下一轮迭代希望当前值加上 α 倍残余量：

$$F^{(k+1)} = F^{(k)} + \alpha r.$$

Jacobi 算子每一步朝更接近解的方向迭代。迭代很多步后，补全结果较好。Jacobi 迭代能较高效收敛到某个位置，但不保证完全解干净方程。好处是不用担心矩阵是否满秩、是否可逆，通过迭代总能求到比较满意的解。

后面还会讲到相关内容，但不需要记忆，比较超纲。若对图形学和物理仿真深入内容感兴趣，这些学科提出了很多高效求解 $AX = B$ 的算法。对本课程，不需要掌握这些。

求解 $AX = B$ 是经典问题，有很多手段。其他课可能讲直接求解，例如 LU 矩阵分解。图形学中常见迭代求解，因为迭代手段较容易做 GPU 并行。可以雅各比迭代，也可以 Gauss-Seidel 迭代。图形学中为加速还衍生多层网格算法：例如 A 是 256×256 矩阵，先在 2×2 矩阵上解，再解 4×4 ，再解 8×8 ，逐层优化 residual。这是高效大规模矩阵求解手段。这些比较超纲。

更关心的是：如果能好好构造 $AX = B$ ，调用现有算法，就能完成图像补全。例如挖一些小鸭子、鸳鸯，补在湖面对应位置，根据算法能算出较好的图像补全。因为完全从数学角度出发，能理解做了什么：把微分信息呈现在图像上。当前景本身背景和新背景比较接近时，融合效果最好，最忠实还原图像。

但如果背景差异比较大，因为只保留微分信息，可能导致整张图相对原图过亮、过暗或偏黄，存在低频系统误差。微分是比较高频的信息。

图像补全就讲这么多。可以尝试手机上的 AI 补全算法。现在已不再使用这些传统方法，而是使用大模型学出的背景填补手段。但传统方法有助于理解。

泊松图像补全是全局问题。对于一个小的区域，边界信息已经确定，内部每个像素值通过整个矩阵同时联立求解最终确定。因此一个像素值跟整个区域非常相关。这就是为什么它是全局的、不能孤立解决的问题。

TwT这里好难

十三、抠图

接下来讲抠图。会多次提到不透明度，即 alpha 通道。

过去，例如把手放在某张图像上，通常先给手印一个 matte，然后确定 0/1 的 mask 掩码。有了 0/1 掩码，保留 1 的部分，替换 0 的部分，完成图像遮罩操作。这也是 mask 名字由来的过程。

过去硬件来的掩码多为 0/1。现在需求过渡到不透明度掩码：背景整个想换掉，所以是 0；前景整个想保留，所以是 1；0 和 1 之间希望有平滑过渡。这通常体现在毛发区域、半透明服装、皮肤边缘等半透明情况。因此需要 0 到 1 之间的过渡不透明区间。

抠图想做的事情：拿到一张图，获得一个 matting，前景为 1，背景为 0，中间是 0/1 过渡。用 matting 乘以原图，把前景完整保留，并具备 alpha 通道，使它可跟后续各种背景混合。这是图像抠图最想实现的事情。

数学上，希望存在 alpha 通道，它乘以前景，加上 $1 - \alpha$ 乘以任意背景，合成抠图图片：

$$C = \alpha F + (1 - \alpha)B.$$

公式简单，但求解困难。图像有 RGB 三个通道，这其实是病态问题。

抠图任务最开始拿到的是一张已经混合的照片。所有 alpha 通道未知，已知的是混合图片的 RGB，即等号左边三个量已知。等号右边前景三个量未知，alpha 未知，背景 B 也全部未知。未知量远大于已知量，因此非常难办。

传统抠图手段有绿幕抠图、蓝幕抠图等，是抠图任务下的特殊状态。因为未知量太多，所以做限定：例如图片是在绿幕前拍摄，背景完全已知，并且背景某两个通道为 0，另一个通道有值。蓝幕抠图中，蓝色通道有颜色。背景被大大砍掉，但即使砍掉两个通道，未知量仍大于已知量，因此做更多限定。例如假设前景一点蓝色都没有。这样得到新方程。

蓝幕抠图方程中，未知量有 F_R, F_G, F_B, α ，已知量有 C_R, C_G, C_B 和背景 B 。蓝幕需要是已知量，即需要给背景单独拍一张照片。它可能有各种蓝色像素，但都是已知量。于是可以解方程，解出 alpha 通道。解出 alpha 后，乘以原始图片，把前景拿出来。拿到前景后，用 alpha blending 方式叠加任意背景。

蓝幕抠图会带来问题：服装会泛蓝。这涉及更高级渲染内容。真实世界光照不仅有直接反射折射，还有物体之间相互映射。绿色会让整个前景偏绿。把物体移到任何背景上时，某些区域可能显得更绿。这是当时抠图技术的不足。

现在抠图技术有具体进展，习题也有相关内容。今天图像处理讲这么多。提前一点下课，有问题可以交流。

LESSON 3 2026-09-21 第1-2节

一、课程导入与内容安排

同学们早上好，感谢大家依旧以极大的热情来参加早八的课程。先给大家一个提示：明天下午的课程会有一次现场签到，算是送给大家早八的一个福利，提醒大家明天来上课。

我们先回顾一下之前的课程内容。到目前为止，我们还处于课程的基础环节。之前我们从显示原理引入了图像的概念，并且讲解了图像处理的相关内容。

从今天开始，我们进入画图的部分。这里先说明两个方向的区别：

- **图像处理**：一般是针对一张已有的图像，从中提取连续语义上的内容。比如从离散的像素中识别出边、角等特征，本质是从离散信号恢复连续的语义，是计算机视觉入门的基础。
- **图形学**：核心思想是你在脑中已经定义好了一条曲线、直线或形状，希望在离散的屏幕空间中将它呈现出来，本质是把连续的几何对象在离散空间中进行表示。我们今天讲的画图 (Drawing)，也叫2D图形学，就是图形学的入门内容。

二、光栅化绘制的背景与核心需求

2.1 显示硬件的演进：从矢量到光栅

画图最基础的任务就是画一条直线。这个问题看起来简单，但我们会花较长的篇幅讲解，也是大家习题中需要实现的内容。

之前讲显示原理时提到过，最早的显示器是阴极电子束成像的矢量显示器。对于矢量显示器来说，硬件指令只需要指定从某一个点划到另一个点，硬件会控制电子束的运动，激活屏幕上对应区域的荧光粉，自然形成一条直线。在那个年代，画线是由硬件自动完成的。

后来出现了光栅化显示器，显示控制方式变成逐点扫描每一个像素位置。光栅显示器的分辨率是固定的，因为它必须以固定的频率刷新屏幕才能保证帧率（FPS）。由此，硬件层面第一次出现了需求：如何把两个点之间连续的直线，用离散的像素点组合呈现出来。

2.2 帧缓冲区与光栅化流程

随着硬件发展，显示器有了专属的存储区域——**帧缓冲区（Frame Buffer）**。有了帧缓冲区之后，每一次刷新屏幕，本质上都是在帧缓冲区上完成各种绘图操作。后来图像存储也沿用了类似的思路，位图文件存储的就是固定分辨率的图像数据。

这就是光栅化从硬件到软件再到存储的完整流程。

2.3 画线操作的普遍性与性能要求

画线操作在今天的显示器中依然占有极大的应用比例。大家可以想象，无论是 1920×1080 还是4K分辨率的显示器，每个像素的绘制都依赖画线操作。

Windows最著名的创造之一就是多窗口UI交互。屏幕上同时运行着PowerPoint、Word、画图等不同的应用程序，每个程序都要在自己的窗口区域内完成绘制。对于帧缓冲区来说，这是一个逐层覆盖的过程：先绘制桌面的所有像素，再绘制PowerPoint窗口，最后绘制最上层的Word窗口和其中的线条。

可以说，画图操作成千上万次地出现在屏幕显示的全过程中。这就要求画线算法必须在非常高效的环境中完成。效率是图形学领域的普遍要求，因此我们这节课的任务不只是讲解如何画直线，更重要的是讲解如何高效地完成直线绘制。

2.4 扫描绘制的核心目标

画图也常被称为**扫描（Scan）**，因为绘制过程和屏幕扫描线的硬件需求高度对应。扫描绘制就是专门针对屏幕扫描做优化的画图方式。

对于画线算法，我们有两个核心需求：

1. **速度越快越好**：这是最核心的性能要求。
2. **尽可能准确**：因为离散化必然存在误差，我们的目标是在当前分辨率下，尽可能准确地还原连续的直线信号。

三、直线绘制的基础算法

3.1 暴力算法及其缺陷

最朴素的思路是暴力求解：遍历屏幕上的每一个像素，检查它是否在直线上。比如 1920×1080 的屏幕，就检查所有两百万个像素。

这种方法一定能得到结果，但资源消耗和时间开销都非常高。我们不希望每次刷新屏幕都遍历所有像素，因此画线算法的优化方向很明确：只修改需要改变的像素，只计算必要的像素位置，执行最直接的运算。

3.2 朴素迭代算法

我们从实际的画线场景出发，先做一些基础设定。图形学中一般需求是画线段，我们可以给自变量一个范围。为了方便，我们先把坐标归一化到 $0 \sim 1$ 区间，0对应左下角像素，1对应右上角像素，实际实现时再根据屏幕分辨率映射到具体的像素坐标。

直线的方程是连续形式的 $Y = MX + B$ 。最直观的像素化操作是：沿着自变量X的方向逐个遍历，对每个X计算对应的Y值，将Y四舍五入到最近的像素位置，然后点亮该像素。

这里就产生了第一个问题：应该顺着X轴遍历，还是顺着Y轴遍历？

- 如果直线方程写成 $Y = MX + B$ ，顺着X遍历会更简单。
- 也可以把方程变形为 $X = f(Y)$ ，顺着Y遍历。

两种方式绘制的像素数量是不同的。因此我们有第一个优化原则：**选择增长更快的方向作为迭代的主循环方向。**

- 当斜率绝对值小于 45° ($|M| < 1$) 时，X方向增长更快，沿X轴遍历，保证不会遗漏任何像素。
- 当斜率绝对值大于 45° ($|M| > 1$) 时，Y方向增长更快，沿Y轴遍历。

图形学的接口都是按这个逻辑设计的，画线的输入规范就是起点坐标 (X_0, Y_0) 和终点坐标 (X_1, Y_1) 。当然，连续坐标经过四舍五入映射到整数像素坐标，会损失一部分精度，但在高分辨率下这个误差可以接受，而固定的API格式能让所有应用更直接高效地调用。

3.3 朴素算法的性能分析

朴素迭代算法已经比暴力算法优化了很多——它只触碰真正需要绘制的像素。但如果精益求精，就会发现它依然很慢，问题出在计算环节：

- 每次迭代都需要做一次浮点乘法 $M \times X$ ，再加上浮点加法。
- 最后还要做一次四舍五入运算。

其中浮点乘法是开销最高的操作。后续的优化方向，就是逐步把这些昂贵的运算替换掉，甚至最终完全消除浮点运算。画线算法是图形学中被优化到极致的经典案例。

四、DDA数字微分分析仪算法

4.1 优化思路：乘法转加法迭代

图形学中非常经典的一种优化手段：如果循环内部存在和迭代变量相关的乘法，就可以利用乘法是加法的累计这一性质，把乘法转换成加法迭代。这也是我们第一步优化的核心思想——把浮点乘法优化掉。

这种优化通常需要额外存储一些历史信息。我们的思路是：既然已经算出了当前X对应的Y值，那么下一个X+1对应的Y值，不需要重新用乘法计算，只需要在之前的结果上累加一个增量即可。

我们来推导一下：

已知当前点的Y值为 $Y_x = M \cdot X + B$

那么下一个点的Y值为：

$$\begin{aligned} Y_{x+1} &= M \cdot (X + 1) + B \\ &= M \cdot X + B + M \\ &= Y_x + M \end{aligned}$$

这样就消除了循环内的乘法运算，每次迭代只需要做一次浮点加法。

4.2 算法步骤

DDA (Differential Digital Analyzer, 数字微分分析仪) 算法的步骤如下：

1. 输入起点 (X_0, Y_0) 和终点 (X_1, Y_1) ，计算斜率 $M = \frac{Y_1 - Y_0}{X_1 - X_0}$ 。

2. 以X为主循环方向 ($|M| < 1$ 时), 从X0开始遍历到X1。
3. 初始Y值为Y0, 点亮对应像素。
4. 每次迭代中, Y累加斜率M, 四舍五入后点亮对应像素。

DDA算法已经避免了循环内的乘法, 整体性能提升很多。这个算法的名字来源于它的核心思想: 通过微分步进的方式, 把乘法转换为加法。

4.3 仍然存在的问题

DDA算法依然存在浮点运算, 每一步都需要做浮点加法和四舍五入。在早期CPU算力稀缺的年代, 浮点运算单元非常少, 整数运算单元更多, 因此人们希望进一步优化, 完全消除浮点运算。

五、布雷森汉姆 (Bresenham) 直线算法

5.1 核心观察: 像素步进的二选一

我们先观察直线光栅化的结果: 当斜率在0到1之间时, 每向右走一个像素 ($X+1$), Y方向只有两种选择——要么保持不变, 要么向上走一格 ($Y+1$)。

也就是说, 每次迭代X必然加1, Y要么加0要么加1。这本质上是一个累积的判断过程, 和四舍五入的逻辑一致。这意味着我们有可能只用整数运算就完成整个画线过程。

接下来我们一步步推导, 如何从DDA算法进化到全整数的布雷森汉姆算法。

5.2 中点判定法与隐函数思想

我们的核心问题转化为: 每次X加1后, 直线上的真实点, 离 $(X+1, Y)$ 更近, 还是离 $(X+1, Y+1)$ 更近? 离哪个像素更近, 就点亮哪个像素。

这个距离判断可以转化为中点判定: 取两个候选像素的中点 $(X+1, Y+0.5)$, 判断这个中点在直线的上方还是下方。

- 如果中点在直线下方, 说明直线更靠近上方的像素 $(X+1, Y+1)$, 应该点亮它。
- 如果中点在直线上方, 说明直线更靠近下方的像素 $(X+1, Y)$, 应该点亮它。

这个判定可以用隐函数来实现。直线可以写成隐式方程 $F(x, y) = 0$ 的形式。对于直线上的点, 函数值等于0; 对于直线上方的点, 函数值符号为负; 对于直线下方的点, 函数值符号为正。我们只需要判断中点代入隐函数后的正负, 就能决定点亮哪个像素。

隐式表达: 不写成 $Y = F(X)$ 的显式形式, 而是写成 $F(X, Y) = 0$ 的形式。隐函数的表达能力强于显函数, 比如圆等几何图形更容易用隐函数表示。

5.3 构造整数隐函数

我们希望隐函数的计算尽可能避免浮点运算, 因此选择用向量点乘的方式构造隐函数。

已知直线的起点 $P_0(X_0, Y_0)$ 是整数坐标, 直线的切向量为 $\vec{T} = (DX, DY)$, 其中 $DX = X_1 - X_0$, $DY = Y_1 - Y_0$, 都是整数。

直线的法向量可以通过切向量旋转 90° 得到, 即 $\vec{N} = (DY, -DX)$ 。这个法向量不需要归一化, 因为我们只关心正负号, 缩放法向量不会改变正负判定的结果, 并且它的分量全是整数。

对于任意一点P，我们构造判别函数：

$$F(P) = F(P) = \overrightarrow{PP_0} \cdot \vec{N}$$

这个函数的正负对应点P在直线的哪一侧：

- $F(P) = 0$ ：点在直线上
- $F(P) > 0$ ：点在直线下方
- $F(P) < 0$ ：点在直线上方

课间修正：之前PPT上的正负对应关系有误，这里更正：当点乘结果大于0时，P在直线下方，此时应该点亮上方的像素；当点乘结果小于0时，P在直线上方，应该点亮下方的像素。

5.4 判别式的迭代更新

和DDA的思路一样，我们不希望每次都重新计算点乘，而是用迭代的方式更新判别式。

假设我们已经算出了当前点P的判别值 $F(P)$ ，现在要计算下一个待判定的中点 $P+\Delta$ 的判别值。根据点乘的性质：

$$F(P + \Delta) = F(P) + \vec{\Delta} \cdot \vec{N}$$

也就是说，每次移动后，判别值只需要加上一个固定的增量。这个增量可以提前预计算，循环内只需要做加法。

现在我们的判定点是中点 $(X+1, Y+0.5)$ ，其中的0.5会引入浮点运算。我们可以利用法向量可以任意缩放性质，把法向量乘以2，这样中点的0.5就会被抵消，整个计算就全部变成整数运算。

我们使用 $2\vec{N} = (2DY, -2DX)$ 作为新的法向量，此时判别式的增量全部为整数。

5.5 算法推导与迭代过程

我们从起点开始一步步推导迭代过程：

1. 起点 (X_0, Y_0) 在直线上，因此初始判别值 $F_0 = 0$ 。
2. 第一个待判定的中点是 $(X_0+1, Y_0+0.5)$ ，代入得：

$$F_{\text{test}} = F_0 + 2DY \cdot 1 + (-2DX) \cdot 0.5 = 2DY - DX$$

这就是初始的测试判别值，是一个整数。

3. 分支判断：
 - 如果 $F_{\text{test}} < 0$ ：中点在直线上方，直线更靠下，点亮右侧像素 $(X+1, Y)$ 。
下一个测试点向右平移一格，判别值更新为： $F_{\text{test}} += 2DY$
 - 如果 $F_{\text{test}} \geq 0$ ：中点在直线下方，直线更靠上，点亮右上像素 $(X+1, Y+1)$ 。
下一个测试点向右上平移一格，判别值更新为： $F_{\text{test}} += 2DY - 2DX$
4. 循环执行，直到X到达终点。

5.6 算法伪代码

布雷森汉姆直线算法 ($0 \leq M \leq 1$ 情况) 的伪代码如下：

输入：起点 (X_0, Y_0) ，终点 (X_1, Y_1)

```

DX = X1 - X0
DY = Y1 - Y0
F = 2 * DY - DX // 初始判别值
Y = Y0
|
for X from X0 to X1:
    点亮像素(X, Y)
    if F >= 0:
        Y = Y + 1
        F = F + 2 * DY - 2 * DX
    else:
        F = F + 2 * DY

```

整个循环内只有整数加法和比较操作，没有任何浮点运算，也没有乘法，性能达到了极致。

5.7 算法意义

布雷森汉姆算法是图形学非常经典的优化案例。它利用隐式方程和迭代思想，把一个看似必须用浮点运算完成的连续直线绘制任务，完全转化为整数运算。这个算法的思路对后续很多图形学算法都有深远影响。

5.5 算法推导与迭代过程（详细讲解）

我们先明确**大前提**：以下推导针对**斜率 $0 \leq m \leq 1$** 的直线。此时 X 方向是主步进方向（每一步 X 一定 + 1），Y 方向是次步进方向（每一步 Y 要么 + 0，要么 + 1）。

一、核心思想：中点判定法

我们的目标是：每向右走一格（X+1），判断接下来该点亮**右侧像素**（X + 1, Y），还是**右上像素**（X + 1, Y + 1）。

判断的标准是：**直线上的真实点，离哪个像素中心更近。**

为了简化距离计算，我们引入**中点判定规则**：

取两个候选像素的中点 $M(X + 1, Y + 0.5)$ ，观察这个中点在直线的哪一侧：

- 如果中点在**直线上方** → 说明直线位置更靠下 → 离下方的右侧像素更近，点亮（X + 1, Y）
- 如果中点在**直线下方** → 说明直线位置更靠上 → 离上方的右上像素更近，点亮（X + 1, Y + 1）

六、直线算法的扩展

6.1 全空间直线的八分区处理

上面的推导只针对斜率在0到1之间的直线。对于其他斜率的直线，我们可以利用对称性处理：

- 斜率大于1时，交换X和Y，沿Y轴方向迭代。
- 斜率为负时，对Y坐标取反，处理完后再映射回来。

通过把平面分成八个卦限，每个卦限都用对称的方式套用上述算法，就可以解决全空间的直线绘制问题，核心逻辑完全一致，只是初始状态和坐标方向略有不同。

6.2 扩展到画圆

布雷森汉姆的思路不仅可以画直线，还可以扩展到画曲线，比如圆。

圆同样可以写成隐式方程 $F(x, y) = x^2 + y^2 - R^2 = 0$ ，同样可以用判别式的正负判断点在圆内还是圆外。和直线类似，画圆时每一步也只有有限的几个候选像素，通过判别式迭代就可以完成绘制。

需要注意的是，圆的斜率从0到360°连续变化，因此我们通常利用八对称性，只绘制1/8个圆弧，剩下的部分通过对称变换得到。可以开8条线程并行绘制这8段圆弧，进一步提升效率。

完整的布雷森汉姆画圆算法超出了本节课的主要范围，但它是非常有趣的思维扩展，感兴趣的同学可以在维基百科上查阅相关内容。

6.3 多边形扫描线算法

除了画线，图形学中另一个重要任务是绘制多边形。扫描线算法就是面向多边形填充的优化算法。

扫描线算法的核心思路是：用水平扫描线从上到下遍历多边形，每一条扫描线只需要确定左右两个边界点，然后直接填充两点之间的所有像素。

左右边界点的更新，本质上就是两个布雷森汉姆画线过程。我们不需要每次都重新计算边界位置，只需要用迭代的方式更新XL和XR，全程都是整数运算，效率非常高。

6.4 凹多边形与三角化

扫描线算法默认处理凸多边形。如果是凹多边形，扫描线会和多条边相交，此时需要用奇偶判定法则：扫描线与边的交点按顺序排列，第奇数个交点到第偶数个交点之间的区域填充，偶数到奇数之间的区域不填充。

但在现代图形学中，已经很少单独处理凸多边形或凹多边形了。更通用的做法是把所有多边形都做三角化处理——任何多边形都可以拆分成若干个三角形。

三角化的优势在于：

1. 三角形一定是凸多边形，算法稳定，不需要额外的凹凸判断。
2. 三角形是现代图形学的基本图元，无论是算法层面还是硬件层面都被深度优化。

现在的显卡都自带硬件三角形绘制加速，你只需要传入三个顶点坐标，硬件就会自动完成整个三角形的光栅化填充，效率远高于软件算法。等到后面讲解渲染、GPU和帧缓冲区的时候，我们还会详细讲硬件并行实现的原理。

课间问答

问：现在浮点运算单元很多，还有并行计算能力，为什么还要优化画线算法？

答：算力永远是稀缺的。并行能力通常用在更粗粒度的任务上，比如同时画100条直线，而不是一条直线上的100个点。每条直线自身依然会用最精简的算法，把并行资源留给更适合并行的操作。即使在今天，单条直线的极致优化依然有意义。

七、颜色插值与图像变形

7.1 颜色线性插值

上面讲的都是几何坐标的光栅化，实际绘制时我们还需要处理颜色。屏幕是RGB颜色空间，绝大多数图形都不是纯黑的，而是带有颜色的，甚至同一个三角形的三个顶点可以有不同的颜色。

要实现顶点颜色的平滑过渡，只需要做颜色插值。最基础的是线性插值：已知两个端点的颜色值，中间任意一点的颜色按距离比例混合。

对于RGB三个颜色通道，每个通道独立做线性插值即可，这是非常简单的操作。

7.2 双线性插值

如果是四边形区域，四个顶点都有颜色值，要计算区域内任意一点的颜色，常用的方法是**双线性插值**。

- 第一步：先对上下两条边分别做线性插值，得到两条边上对应点的颜色。
- 第二步：对这两个点再做一次纵向的线性插值，得到最终点的颜色。

简单来说，就是通过点插值出直线，再通过直线插值出平面，这就是双线性插值的过程。

7.3 图像变形 (Image Warping)

我们在界面中经常会遇到图像变换的场景，比如拖动窗口、缩放、旋转，还有一些非线性的窗口动效。这些操作本质上都是**图像变形**。

图像变形和插值的原理是相通的，只不过插值的对象从颜色空间变成了坐标空间：

- 图像本身有自己的**UV坐标空间**，通常归一化到(0,0)到(1,1)之间，这是图像的原始规范空间。
- 我们要把图像绘制到屏幕的XY坐标空间，四个顶点对应到屏幕上的四个位置。
- 对于屏幕上的任意一个像素，我们通过坐标插值反算出它对应的UV坐标，再从原图中取出对应位置的颜色，绘制到屏幕上。

整个过程的核心是坐标空间的插值，颜色只是坐标索引的结果。这也是为什么我们把图像变形放在图形学的画图部分讲解，而不是图像处理部分——它本质上是几何坐标操作，而非像素值操作。

7.4 交互式形变的实现

图像变形在图像处理软件中有非常广泛的应用，比如Photoshop中的变形工具。

为了方便用户交互，通常的做法是在图像上覆盖一层网格，用户可以拖动网格上的控制点来改变图像形状。为了让操作更自然，还发明了笔刷式的交互：笔刷中心的位移权重最高，周围按半径逐渐衰减，用户拖动笔刷就能让一大片区域跟着变形，实现非线性的形变效果。

7.5 四边形插值的两种方式

这里补充一个重要的观察：四边形的线性插值并不是唯一的，有两种典型的实现方式，结果不同：

1. **双线性插值**：先沿水平方向插值，再沿垂直方向插值。当四边形是矩形时，过渡非常均匀；但当四边形被拉伸变形时，对角线方向会出现扭曲。
2. **三角剖分插值**：先把四边形沿对角线拆成两个三角形，在每个三角形内部做线性插值。

两种方式没有绝对的对错，适用场景不同：

- 如果是三维物体的透视投影变形，双线性插值的结果可能更符合真实透视。
- 如果是规则的网格变形，三角剖分的结果可能更符合预期。

在图形学中，最终的渲染结果和输入的表达形式高度相关。如果输入是三角形网格，就会走三角形插值的管线；如果输入是四边形网格，就会走双线性插值的管线。具体选择哪种，要根据实际应用需求决定。

八、曲线内容展望

8.1 曲线的表达需求

讲完直线，我们自然会扩展到曲线。曲线在图形学、艺术、设计和工业领域都有非常广泛的应用，有强烈的表达需求。

艺术领域，梵高的画作中就用大量旋转、湍流的曲线来表现云朵、植物等元素，传递出独特的生命力。西班牙建筑师安东尼·高迪有一句名言：“直线属于人类，曲线属于上帝。”他的作品，比如米拉之家、圣家族大教堂，大量使用贴近自然的曲线构造，极力避免火柴盒式的现代建筑。

无论是还原真实世界，还是艺术表达，我们都需要描绘曲线。图形学已经很好地解决了曲线的表示和光栅化问题。

8.2 矢量图与曲线

图像分为矢量图和光栅图两大类。在矢量图格式中，曲线是最核心的表达内容，最典型的就是SVG格式。Logo设计、字体设计最终通常都会用SVG格式存储，因为矢量图可以任意缩放而不失真。

曲线最终呈现在屏幕上，都需要经过光栅化。和直线一样，我们可以用隐函数结合布雷森汉姆的思路把曲线光栅化。但曲线的非线性更强，低分辨率下信息损失更多，因此用矢量格式保存曲线的原始定义，在需要时再光栅化，是更合理的方式。

8.3 工业领域的曲线应用：CAD/CAE/CAM

曲线不仅用于艺术和设计，还广泛应用在工业领域，也就是CAx（计算机辅助技术）领域，主要包含三个方向：

1. **CAD（计算机辅助设计）**：核心需求是设计出优雅、流畅的曲线曲面。比如汽车的流线型外形，既要美观也要符合空气动力学。CAD中的几何表达都是连续的曲线曲面。
2. **CAE（计算机辅助工程）**：核心需求是物理仿真，比如结构力学、温度扩散、流体流动等。这些问题都需要求解复杂的偏微分方程（比如流体的纳维-斯托克斯方程、热扩散方程）。连续的几何体无法直接求解，必须离散化成网格（Mesh），比如三角形网格。离散化的精度、网格的疏密分布，直接决定了仿真的误差和效率，这是CAE领域非常重要的研究方向。
3. **CAM（计算机辅助制造）**：核心需求是把设计好的零件制造出来。无论是切割、模具还是3D打印，刀具的运动轨迹往往是线性的，因此CAM是一个混合了连续与离散的领域——既要忠实还原CAD的连续曲面设计，又要用线性的刀具路径去逼近它。

感兴趣的同学可以去GrabCAD网站看看，上面有非常多的3D CAD设计模型，广泛应用在Solid-Works、AutoCAD等经典设计软件中。

8.4 后续课程预告

2D图形学的曲线有很多种表达，比如圆、隐式方程、折线，其中我们重点讲解的是贝塞尔曲线。关于曲线的详细内容，我们留到明天下午的课继续讲解。

今天的课就到这里，谢谢大家。

课后说明：平时成绩占5分，评分主要看大家作业的实现效果。除非有大问题，一般不会扣分。

LESSON 4 2026-09-22 第5-6节

课程导入

上节课我们讲解了直线扫描算法，以及2D图像图形学的基础，同时初步介绍了曲线的方向，包括曲线的广泛应用背景，以及在CAD、CAE、CAM中的具体需求与区别。

今天我们从曲线开始，讲解与图形学更相关的2D曲线基础内容。上节课我们已经提到，曲线分为很多类别，包括显式曲线、隐式曲线、分形，以及我们会重点讲解的贝塞尔曲线。接下来我们逐一讲解这些曲线的内容。

显式曲线

我们先从显式曲线讲起。最简单的显式曲线，就是把很多直线段连接在一起，构成一条非直线的线，这就已经是一条曲线了。显式表示通常比较容易编辑和操作。

从定义上来说，显式曲线一般写成 $y = f(x)$ 的形式。它既可以是分段的形式，每一段都是 $y = mx + b$ 的直线表达式，也可以是更复杂的非线性函数，比如 $y = x^2$ 。

在图形学中，尤其是在表达几何的相关方向，显式和隐式是非常常用的概念，它们界定了方法本质上的特性。提到显式，很多人会想到离散、不连续，或者点云这类存储格式，但这些并不是显式最本质的区别，只是它常表现出的优缺点。

更本质地说，显式是直接定义为 $y = f(x)$ 形式的表达方式，可以用于曲线表达，也可以用于空间几何体表达。这种表达最大的特点是，可以直接代入公式计算，得到想要的y值。比如想知道点云在空间中的位置，就可以直接找到点云坐标进行插值，也就是可以直接显式地计算。

显式的优势很明显：需要取值的时候，代入公式就能快速求出结果。但它的不足也非常突出：要把复杂形状表示成显式形式往往并不容易。比如圆的显式表达，相对来说就会复杂一些。

隐式曲线

与之相对的隐式曲线，应用会更普遍一些，因为它的表达能力更强。我们可以把它抽象为 $f(x, y) = 0$ 的函数形式，包含的范围很广，比如常见的解析方程。以圆为例，用隐式形式可以非常自然地表达出来。

很多人提到隐式时，会认为它是连续的，不是离散形式。这确实是它的一个特性——隐式表达通常描述的是一个连续的对象。但这个连续的对象也可以存储在离散空间中，比如可以用规则网格，在每个网格上存储数值，通过这些数值定义一个空间。只要空间本身的定义是一个函数，确定每个点的值需要求解 $f(x) = 0$ 的方程，那么这个空间就可以是连续的隐式方程。

讲到几何建模时，我们会更多地讲解几何表达，其中**符号距离函数（Signed Distance Function）**就是后面要讲到的一种重要的隐式表达方式。

除了直接写出解析解、通过距离场反向求解的形式外，还有经典的隐式表达例子，比如分形。分形相对复杂，不是本课程的重点，但它比较有趣。分形是通过无穷迭代，每一层按规则重复结构，最终达到极限时组成的确定形状。只不过在有限的计算时间内，很难完成这样的绘制。分形可以是连续的，但很难做到光滑，它属于隐式能够表达的内容，可以表达非常复杂的曲线。

参数化曲线

参数化曲线的定义与优势

显式和隐式各有优缺点。显式的优势是可以高效计算某点的值，但表达能力有限；隐式的表达能力很强，但每次取值都必须求解方程，很多表达特性不直观。

因此很自然地，人们需要一种介于二者之间的表达方式，由此产生了**参数化（parametric）**表达。

需要说明的是，隐式和显式的定义在领域内相对模糊，很难明确划分二者的界限，因此参数化到底属于隐式还是显式，也没有明确的定论。我们先写出参数化的形式。

参数化曲线通常以参数 t 作为自变量， x 和 y 共同随 t 变化，写成：

$$\begin{cases} x = f(t) \\ y = g(t) \end{cases}$$

我们可以用经典的例子来理解。之前提到圆用显式表达会比较复杂，但用参数化的方法，就很容易写成关于参数 t 的 x 、 y 表达式。

这样一来，参数化继承了所有显式的优势：可以随机采样，在任意 t 值处采样，就能高效计算出 x 、 y 坐标。这也是目前更多人倾向于认为参数化表达属于显式范畴的原因。

圆很容易用参数化表达，这个优势还可以进一步扩展。圆的解析方程大家很熟悉，但如果要表达螺旋线，找它的可解析方程就会相对抽象。但如果用参数化表达，这件事就会变得容易：在角度旋转的同时，圆的半径逐步增长，只要把半径变成与 t 相关的表达式，就能扩展成螺旋线的表达。

由此我们可以感受到参数化曲线的优势：首先，它的表达能力远超一般的显式表达；同时，它的计算也非常高效。

为了感受参数化表达的优势，非常推荐大家尝试各类工具软件，比如Desmos，它支持通过简单的交互编写参数化方程。你只要敲入参数，它就可以把这些参数变成用户可控的变量，动态感受参数变化对曲线最终结果的影响。这类软件也体现了参数化曲线易于控制、易于编辑的优势。

我们可以举更多例子，比如各类著名的曲线，以及后面会重点讲解的贝塞尔曲线——也是大家实验课需要实现的内容。总的来说，参数化曲线就是根据某个参数 t ，对应变化的一组表达形式。

参数化曲线还有另外一个优势：它很容易扩展到三维，甚至可以扩展到更高维度。比如螺旋线扩展到三维，就可以得到空间圆柱螺旋线，大家也可以在Desmos上尝试。这类曲线设计在建筑等领域非常有用。

我们来总结一下参数化表达的优势：

1. 容易扩展到三维；
2. 可视化非常容易，只要确定参数范围（比如0到1），在范围内均匀取点，就可以完成可视化绘制；
3. 便于求解微分。曲线和直线最大的不同是非线性变化，我们通常非常关心曲线的微分变化。对于显式表达 $y = f(x)$ ，切线的 dy/dx 比较好计算；对于参数化表达，可以借助参数 t ，快速得到随 t 变化的切线微分信息，计算同样高效。

多项式曲线及其不足

参数化方程和隐式方程有一点相似：表达能力强的同时，表达形式通常不唯一。以圆为例，常见的参数化方程是一种形式，但参数可以自行选择，也可以选特殊的量作为参数。任何满足显式方程的解，都可以作为参数化表达，但其中有些形式更友好，有些会带来理解上的困难。

接下来我们介绍几种常见的参数化曲线形式。首先是最容易理解、但并不那么通用的**多项式（polynomial）曲线**，我们会解释它为什么不常用。后面还会讲到贝塞尔曲线、B样条曲线等。这些曲线最终的表达能力可能是相同的，比如都可以表达圆形，但因为各自的优缺点不同，最终流行开来的是某一种，而不是另一种。曲线表达方式的优劣，和实际使用场景有非常大的关系。

多项式曲线形式简单，从数学角度非常容易理解，最开始受到人们的关注，但它存在明显的不足。

我们举一个实际应用的例子：空间中有一些点，想知道这些点定义了怎样一条曲线。这是真实世界中常见的应用场景：比如手绘了一条复杂曲线的草稿，描了一些散点，想要恢复曲线的形状；又比如扫描真实建筑物的点云，希望从点云恢复背后连续的曲线。

这时我们可能会考虑，它会不会是一条多项式曲线。如果是多项式曲线，点的数量多、自由度高，就可以尝试用高次多项式曲线去拟合这些点。

我们可以列一个自由度和点数一致的高次多项式方程，把所有点的坐标代入方程，反向求解所有系数，就能得到一条满足所有点都在曲线上的曲线。

多项式曲线的优势在于：

1. 高次方程是可求解的；
2. 保证曲线经过所有给定的点；
3. 高次多项式曲线足够光滑连续，可以求多阶微分。

但它的不足也非常明显。把曲线全貌展示出来就会发现，在定义范围之外，以及边界区域，曲线的形状会严重偏离预期，和最初几个点定义的曲线有非常大的差异。

这体现了高次多项式表达的不足：当幂次较高时，边界经常会出现**过冲（Over shooting）**的情况；同时，反向求解高次方程的过程也比较复杂。而如果不采用高次多项式，又很难保证曲线一定穿过所有点。这体现了控制能力和表达复杂性之间的冲突。

分段曲线的思路

更通用、更被广泛选择的参数化曲线，是分段化的曲线，也是我们后面主要学习的内容。比如三次样条曲线，就广泛应用在各个领域。我们课程最开始介绍的字体字形设计，就大量使用了贝塞尔曲线，其中也包含直线段。

答案很明显：如果使用全局的多项式参数化表达，参数数量多、难以控制。我们的选择是把它切断，做分段化处理。

分段化之后，每一小节只需要以足够的自由度，表达周围的几个点就够了。通过分段化，降低了每一段的任务难度，曲线就可以用较低的阶数光滑表达每一段的内容。

整体通过分段，依然能保持全局足够的表达能力，去表达各种各样的曲线，同时也更符合人的理解习惯。字体中的字形就是由分段的贝塞尔曲线和直线组合而成，最终得到非常好用的参数化表达。

分段是一件好事，但也带来了新的挑战：分段之后，首先需要保证线段之间是连接的，这是最基本的C0连续性；除此之外，还会有更高阶的连续性要求。

比如我们可以规定分段的端点必须重合；在此基础上，还希望左右两边的多项式参数化表达，在端点处的微分方向和大小都具有连续性，让整体看起来更加光滑。

接下来我们会着重讲解两类曲线：贝塞尔曲线和样条曲线，重点关注它们在连续性和可控性两方面的表现，以及如何平衡二者。

插值与基函数基础

讲解这些曲线之前，最基础的知识是**插值（interpolation）**。各类曲线大量使用插值能力，不断提高插值的阶数。我们先从插值的基础讲起。

（注：）线性插值（英文常称 lerp，全称 Linear Interpolation），最简单的理解就是：已知两个点的数值，在它们之间连一条直线，算出这条直线上任意一个位置的数值。因为插值的轨迹是直线，所以叫“线性”插值。

线性插值

插值我们之前已经讲过，线性插值（lerp）是所有插值里最简单的形式。

比如有两个值 u_0 和 u_1 ，用一个线性核，在 u_0 和 u_1 之间进行线性插值。线性插值的好处是，插值结果一定在 u_0 和 u_1 之间，并且满足一定的连续性。

把插值扩展到二维的xy坐标，点与点之间分段进行线性插值，就能得到最基础的折线。

另外值得一提的是，插值天然符合参数化曲线的需求。插值过程中天然产生了参数 t ，调整插值权重的量，正好就是决定参数化表达的参数。

1. 最基础：一维数值的线性插值

比如我们有两个已知的数值 u_0 和 u_1 ，想求它们之间某个位置的值，就引入一个参数 t （取值范围是 $0 \leq t \leq 1$ ）：

- $t = 0$ 对应起点，结果就是 u_0
- $t = 1$ 对应终点，结果就是 u_1
- t 在 0 到 1 之间，就对应两个点之间的位置

计算公式是：

$$\text{插值结果} = (1 - t) \times u_0 + t \times u_1$$

举个具体例子： $u_0 = 10$, $u_1 = 20$

- $t = 0.5$ （正中间）：结果 $= 0.5 \times 10 + 0.5 \times 20 = 15$
- $t = 0.2$ （离起点更近）：结果 $= 0.8 \times 10 + 0.2 \times 20 = 12$

其中 $(1 - t)$ 和 t 就是两个值的**权重**：离哪个点越近，哪个点的权重就越大，权重随 t 匀速（线性）变化 —— 这就是之前说的“线性核”。

2. 扩展到二维：坐标的线性插值

把插值用到平面的 xy 坐标上，就是课程里说的“扩展到二维”：

已知两个点 (x_0, y_0) 和 (x_1, y_1) ，对 x 坐标和 y 坐标**分别做线性插值**，就能得到中间任意 t 位置的点坐标：

$$\begin{cases} x = (1 - t)x_0 + tx_1 \\ y = (1 - t)y_0 + ty_1 \end{cases}$$



把一串控制点两两之间都做线性插值、连起来，就是最基础的**折线**。

基函数的概念

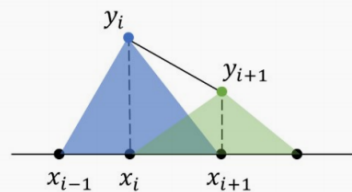
我们对线性插值做一个扩展：插值的本质是在两个值之间选择不同的权重。对于每一对相邻的值 y_i 和 y_{i+1} ，它们都按照权重均匀分布在插值的 x 空间上。

我们可以可视化线性插值的权重分布：在对应点的位置权重最大，值为1；到下一个点的位置，权重衰减到0。从0到1、再从1到0的区间，是线性插值的基础设定。

我们可以把插值的核写成插值函数的形式，它本身往往是分段函数：在每一段内有对应的值，其他部分为0。它的形状是一个小三角形，中间上升、两边下降，两侧为0。

这个插值函数，我们称之为**基函数**（basis function）。基函数是用于插值的权重函数。

Basis function



$$y = \frac{x_{i+1} - x}{x_{i+1} - x_i} y_i + \frac{x - x_i}{x_{i+1} - x_i} y_{i+1}$$

$$B_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}}, & x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{x_{i+1} - x_i}, & x \in [x_i, x_{i+1}] \\ 0, & \text{else} \end{cases}$$

$$y = B_i(x)y_i + B_{i+1}(x)y_{i+1}$$

$$y = \sum_i B_i(x)y_i$$

$$B_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

21

所有的插值过程，都可以写成已知量通过基函数最终累积成目标值的过程。已知量是离散的，基函数把它们延展到连续空间；最终曲线的连续性，很大程度上取决于使用的基函数。如果用线性基函数，最终曲线就是线性连续的情况。

理想的基函数需要满足一些特点：

- 在自身对应的采样点位置，权重正好为1，也就是该点的值就是它本身；
- 在其他基函数对应的采样点位置，尽可能不贡献值。

我们可以用线性基函数，但如果想要更好的连续性，就需要更光滑的基函数。比如高斯基函数，它的分布非常光滑，在自身点位置比重较高，在其他点对应的位置已经衰减到0。用高斯基函数做加权平均，得到的曲线整体更光滑，也更容易通过控制点进行控制。

大家也可以去Desmos上尝试感受各类参数化曲线的易编辑、易控制能力。

插值是图形学中非常重要的概念，因为我们需要用离散空间表达背后连续的对象，同时又希望尽可能高效地操作，插值就成了沟通离散和连续的有效手段。

后面讲到各类曲线时，我们通常都会反过来思考：这条参数化曲线的基函数是什么形式，这往往决定了曲线的很多属性。

(注：)

我们先拆解这句话：

在其他基函数对应的采样点位置，尽可能不贡献值。

每个采样点都有**专属的基函数**，比如采样点 P_0 对应基函数 B_0 ，采样点 P_1 对应基函数 B_1 。

这句话的意思是：

基函数 B_i （属于采样点 P_i ），在**别的采样点** P_j （ $j \neq i$ ）的位置上，它的权重值最好是0，也就是不对这个位置的结果产生贡献。

举线性插值的例子（满足这个理想条件）

两个采样点 t_0 、 t_1 ：

- $B_0(t) = 1 - t$ ：在 t_0 处 = 1，在 t_1 处 = 0（在别人的采样点完全不贡献）
- $B_1(t) = t$ ：在 t_1 处 = 1，在 t_0 处 = 0（在别人的采样点完全不贡献）

插值公式：

$$P(t) = B_0(t)P_0 + B_1(t)P_1$$

当 $t = t_1$ （属于 B_1 的采样点）时， $B_0(t_1) = 0$ ，所以 P_0 不会对这个点产生任何影响，结果正好等于 P_1 。

这就能保证曲线**精确经过采样点**，也就是插值曲线。

为什么原文写“尽可能”而不是“必须”？

有些基函数做不到严格等于0。

比如贝塞尔曲线的伯恩斯坦基函数：它在其他控制点的位置**不等于0**，所以贝塞尔曲线不会穿过中间控制点，只是被控制点“拉着走”，属于**近似曲线**，不是插值曲线。



这就是原文说“尽可能不贡献值”的原因，这是理想基函数的目标，不是所有基函数都能实现。

贝塞尔曲线

【Freya | 熟肉】贝塞尔曲线之美_哔哩哔哩_bilibili

直观认识与构造过程

接下来我们来看贝塞尔曲线。二维贝塞尔曲线由曲线和上方的控制点组成。

大家可能对贝塞尔曲线并不陌生，比如系统自带的画图软件中画曲线，内部使用的就是贝塞尔曲线：先画起点、再画终点，然后设计控制点，就会自动形成曲线的形状。在PowerPoint等Office软件中编辑形状时，很多形状也都是以贝塞尔曲线的方式呈现的。

我们可以直观感受一下贝塞尔曲线的构造：通过拖动起点、终点和控制点，可以改变曲线的形状。贝塞尔曲线通过控制点以插值的方式最终生成。

我们可以通过辅助线直观理解插值过程：比如三个点可以插值出一条线段，另外三个点插值出另一条线段；这两条线段再做一次插值，最终得到的就是贝塞尔曲线。

贝塞尔曲线的优势首先是好控制：可以控制起点和终点的位置，还可以控制端点处切线的方向。比如希望曲线终点处的边和y轴平行，就可以把控制点拖到和y轴一致的方向，曲线就会更贴合y轴；同理，如果希望起点处和x轴平行，也可以通过控制点实现一阶微分的控制。

贝塞尔曲线于1962年在实际工作中被提出，最初的应用场景是汽车设计。当时汽车是昂贵的奢侈品，人们希望它的造型足够经典，同时符合空气动力学；为了方便控制形状，同时保证足够光滑、不产生额外空气阻力，贝塞尔曲线应运而生。

阶数与递归构造

我们来看更数学化的内容。贝塞尔曲线是反复插值的过程，我们把它具体写下来。

先从只有一个控制点的情况开始，方便理解插值过程。曲线的形成过程可以按点的生成来理解：控制点和起点、端点连成两条直线，在起点和控制点之间做线性插值，在控制点和终点之间做线性插值，得到两个中间点。

接下来，对这两个中间点再做一次线性插值。我们可以用动图来感受这个过程：想象有一个小点从起点移动到控制点，同时另一个小点从控制点移动到终点；这两个点移动的过程中，它们之间拉着一根可以自由延伸的弹性绳，弹性绳上又有一个小点，从绳的起点走到绳的终点。这个中间点的移动轨迹，就是贝塞尔曲线的轨迹。

从公式上来说，中间移动的两个点通过线性插值获得，最终形成曲线轨迹的点，由这两个中间点再做一次线性插值获得。

最值得注意的是：每一次线性插值都会引入一个新的参数 t ，而贝塞尔曲线定义所有插值过程共用同一个参数 t 。 t 的取值范围在0到1之间，当 $t=0.6$ 时，在起点和控制点之间插值到0.6的位置，在控制点和终点之间也插值到0.6的位置，最终曲线上的点也对应 $t=0.6$ 的位置。

通过共用唯一的参数 t ，三步插值统一在一起，就完成了0到1区间内贝塞尔曲线的插值。

这里面包含递归的思想：

- 如果只有两个点，连接起来就是直线；
- 如果起点和终点中间有一个控制点，需要先完成第一层的两个线性插值，再完成第二层的插值，得到整条曲线；
- 如果增加更多的控制点，中间的线段每次都会增多，为了最终得到一条轨迹，就需要多一层插值过程。

每一层插值都基于线性插值，每增加一个控制点，就意味着多了一层线性插值，参数方程就增加一个阶数：

- 1个控制点 → 二阶贝塞尔曲线（二次）

- 2个控制点 → 三阶贝塞尔曲线（三次）

原因在于：两个点之间的直线是关于 t 的一次函数；控制点的轨迹是 t 的一次函数，两个一次函数再做线性插值，乘以 $(1-t)$ 和 t ，幂次就升级到二次。因此一个控制点的曲线叫做二阶贝塞尔曲线。同理，每加一个控制点，就多一层线性插值，阶数从2次变到3次。这就是贝塞尔曲线作为多项式曲线，阶数增加的过程。

2. 常见低阶的具体形式

对应课程里的例子：

- **1 阶（直线）**：2 个控制点

$$B(t) = (1 - t)P_0 + tP_1$$

就是普通的线性插值，直线是特殊的贝塞尔曲线。

- **2 阶（二次）**：3 个控制点

$$B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2$$

- **3 阶（三次）**：4 个控制点

$$B(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t)P_2 + t^3 P_3$$

系数 1、3、3、1 就是杨辉三角的第三行，和课程描述完全对应。

代码实现思路

实验课lab1的任务，是写一个迭代算法来实现贝塞尔曲线。大家可以使用AI来辅助完成。

我们讨论一下如何用递归函数实现贝塞尔曲线。对于代码框架来说，比较有挑战的是函数的输入输出应该如何设计。

贝塞尔曲线的API调用已经基本固化：通常将所有控制点作为列表输入，再输入参数 t 的值，函数的任务就是返回这组控制点定义的贝塞尔曲线，在参数等于 t 时的屏幕坐标。

有了这样的定义，递归实现的思路就很清晰：先确定简单情况下的返回结果，再定义递归调用的规则。

这里额外说明一点：上节课我们讲直线扫描时做了非常多的优化，讲到曲线时，我们让大家实现递归的方法。大家可能会觉得递归方法效率不高，但事实上，真实场景中绘制曲线最常用的就是递归方法。

主要原因是：曲线自身的复杂度远超过直线，高次多项式曲线确实可以写出解析式，每个点都可以通过多项式计算。但对于离散浮点数运算来说，直接计算多项式会有很大的精度误差——尤其是控制点和 t 都在0到1之间，会出现很小的数和很大的数相乘的情况，导致浮点数误差，最终产生连续性上的问题。

为了保持较好的连续性，我们依赖插值的方式，而依赖插值就必须递归地完成曲线绘制。

数学形式与伯恩斯坦基函数

接下来我们从解析形式感受贝塞尔曲线的性质，理解它为什么流行、为什么切向容易控制。

二阶贝塞尔曲线的参数化形式展开后，我们可以把它写成基函数的形式，观察贝塞尔曲线的基函数分布。二阶贝塞尔曲线的基函数正好呈现杨辉三角的形状。增加阶数到三阶，基函数也会对应更高一层的杨辉三角形式。

这并不是巧合：因为基函数由一级一级的线性插值生成，每一次插值都在两个参数之间选择 $(1-t)$ 或者 t ，取决于对应的控制点。最终每个控制点的权重，对应选了多少次 $(1-t)$ 、多少次 t ，正好是组合数 $C(n,k)$ 的排列。

因此贝塞尔曲线的基函数展开后，正好是杨辉三角的形式。随着控制点增多，阶数逐渐增高，每一项的幂次都会相应增加。由此我们可以把任意阶数的贝塞尔曲线，写成更通用的数学形式。

这类基函数对应的多项式，叫做**伯恩斯坦多项式 (Bernstein polynomial)**，这个名字也被用来定义这类基函数。

数值稳定性与矩阵形式

我们可以总结一下贝塞尔曲线的计算方式：可以用迭代的方法，也可以用解析的方法。

在基函数中，会频繁出现 $t(1-t)$ 的结构。因为 t 处于0到1之间， t 和 $(1-t)$ 一个接近0、一个接近1，二者相乘会带来浮点数精度的问题，误差容易被放大。

因此直接计算高次贝塞尔曲线的参数形式，浮点数乘法非常不稳定；相对而言，迭代算法往往更加高效、稳定。

除此之外，我们还可以把贝塞尔曲线写成向量与矩阵相乘的向量化计算结构：把参数写成向量，和控制点向量做点乘；或者进一步拆解基函数，提取参数的系数矩阵，写成“向量-矩阵-向量”的乘积形式。

写成矩阵形式的好处，主要是便于向更高阶、更多控制点的曲线扩展，也便于向二维、三维扩展，能帮助我们更好地理解曲线的可扩展性。

从基函数的角度感受：用线性基函数插值得到的是线性插值结果；用三阶贝塞尔对应的三阶基函数做插值，得到的曲线就更加连续。曲线的连续性本质上由基函数决定。

矩阵形式也更方便我们计算一阶微分、二阶微分，分析曲线的法向等性质。

端点与切线性质

贝塞尔曲线，尤其是三阶贝塞尔曲线，使用非常广泛。它能保证经过两个端点，这是插值本身的性质；同时，还可以用另外两个控制点控制两端的切线方向。我们从数学上解释这个性质。

通过矩阵形式对参数 t 求一阶微分，就可以得到端点处的微分表达式。以三阶贝塞尔曲线为例：

- 在起点 $t=0$ 处，切线方向等于起点与第一个控制点连线的方向；
- 在终点 $t=1$ 处，切线方向等于最后一个控制点与终点连线的方向。

更高阶的贝塞尔曲线，在端点处也都保持这样的性质。

这个性质带来的好处是：贝塞尔曲线非常适合作为分段函数中的一段。它能保证经过端点，保证C0连续；还可以通过调整控制点，实现C1连续，也就是切线的连续。

每一段三阶贝塞尔曲线有4个控制点：两个是起点和终点，保证曲线经过这两个点；另外两个是控制点，通过拖动它们可以控制曲线的切向，得到圆润的拐弯，也可以控制曲线的间距。配合直线段，它就是非常适合设计工作的曲线。

如果要想实现C1连续，只需要让相邻两段在连接点处的两个控制点，沿同一方向且距离相等，就能保证该点处切线方向相同、大小相等，完成C1连续。

二、数学严格证明（一阶导数）

参数曲线的切线方向，就是曲线对参数 t 的一阶导数（切向量）的方向。我们对三阶贝塞尔公式求导，代入端点即可验证。

1. 三阶贝塞尔曲线公式

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3, \quad t \in [0, 1]$$

2. 对参数 t 求一阶导（切向量）

逐项求导并整理：

$$\begin{aligned} B'(t) &= -3(1-t)^2 P_0 + 3[(1-t)^2 - 2t(1-t)] P_1 + 3[2t(1-t) - t^2] P_2 + 3t^2 P_3 \\ &= 3(1-t)^2(P_1 - P_0) + 6t(1-t)(P_2 - P_1) + 3t^2(P_3 - P_2) \end{aligned}$$

3. 代入起点 $t=0$

把 $t = 0$ 代入，所有含 t 的项都归零：

$$B'(0) = 3(P_1 - P_0)$$

3 是正的标量，只改变向量长度、不改变方向。因此：

起点处的切向量，与向量 $\overrightarrow{P_0 P_1}$ 完全同方向。

4. 代入终点 $t=1$

把 $t = 1$ 代入，所有含 $(1-t)$ 的项都归零：

$$B'(1) = 3(P_3 - P_2)$$

同理：

终点处的切向量，与向量 $\overrightarrow{P_2 P_3}$ 完全同方向。



连续性：C连续与G连续

对于分段曲线来说，连续性非常重要，在汽车设计等实际应用中都是核心需求。我们把连续性单独总结。

大家比较熟悉的是**C类连续性**，它完全来自数学定义：自变量是参数 t ，因变量是插值得到的曲线坐标。C类连续性描述的是坐标对 t 的各阶导数的连续性。

- C0连续：两个端点重合，位置连续；
- C1连续：一阶导数连续，也就是切线不仅平行，大小也相等；
- C2连续：二阶导数也连续匹配。

但需要注意的是，曲线的几何形状和参数本身没有必然联系。比如同一条曲线，可以用 t 做参数，也可以用 $2t$ 做参数，只要取值范围对应调整，最终几何上的点完全一致。

因此在曲线领域，我们额外提出一组连续性定义，叫做**G连续（几何连续性）**。

G连续和C连续有很多相近的特性：

- G0连续和C0连续完全一致，是最基础的连续，即两个端点相接；
- G1连续只要求切线方向一致，不关心切线的大小（也就是参数变化的速率），比C1连续的要求更宽松；
- G2连续建立在G1的基础上，要求曲率连续。曲率可以理解为曲线某点处外接圆半径的倒数，描述切线随弧长的变化率，是和二阶微分相关的量。

需要说明的是：当参数是弧长时，曲率正好等于二阶微分的模长；如果参数不是弧长，曲率可以通过一阶、二阶微分计算得到。

两类连续性的包含关系：

- C类连续性中，C2连续必然满足C1连续，C1连续必然满足C0连续；
- G类连续性中，G2连续必然满足G1连续，G1连续必然满足G0连续；
- C2连续必然满足G2连续，但跨阶不存在包含关系：C1连续不代表满足G2连续，G2连续也不代表满足C1连续。

我们可以直观理解不同阶的G连续：

- G0：两条曲线在端点处相接即可；
- G1：两条曲线在端点处切线方向相同；
- G2：两条曲线在端点处曲率也连续匹配，切线变化的速率一致，整体更加平滑；
- G3：曲率本身也可微，连续性进一步升高。

工业界很多场景需要G2连续，因为仿真计算会要求几何表面的法线连续，甚至法线的微分也存在。设计领域的实际需求大概在G1到G2之间：纯视觉设计G1用得更多，涉及仿真、偏微分方程求解的场景对G2连续性需求更高。

贝塞尔曲线可以很容易做到G0、G1连续，甚至C1连续；但要做到G2、C2、G3连续，难度就会高很多。如果放开所有自由度，可以调整出满足G2的曲线；但如果入口和出口的控制点是固定的，不能天然保证一定存在满足条件的贝塞尔曲线。

注：对于单段 n 阶非退化贝塞尔曲线，一定满足 $C^1, C^2, \dots, C^n, G^1, G^2, \dots, G^n$ 。上面的“条件”讨论的是多条曲线拼接。

贝塞尔曲面（三维扩展）

贝塞尔曲线非常容易扩展到三维。扩展到三维时，控制点从二维点变成三维空间点，同时引入两组参数 u 和 v ，就可以构成贝塞尔曲面。

比如 4×4 的三阶贝塞尔曲面，有 4×4 个控制点，和二维贝塞尔曲线非常类似：曲面一定会经过四个角的控制点，中间的控制点可以用来控制曲面沿 u 方向和 v 方向的切线，进而控制曲面的切面。

数学上也可以规整地写成 u 、 v 两组参数化形式，中间是控制点和参数矩阵，很容易应用到实际工作中做数学推导。

凸包性与局限性

贝塞尔曲线还有额外的好处：它具有**凸包性**。如果用控制点做一个包围盒，曲线一定落在这个包围盒内部。

这是因为曲线通过插值生成，即使控制点出现交叉的情况，插值结果也一定会落在控制点的包围盒内部。这个局域性特征非常方便计算碰撞检测等相关问题。

接下来我们讲贝塞尔曲线的不足。

贝塞尔曲线是高阶多项式，可以分段连接。但如果修改中间某一段曲线，并且要求保持连续性，就会影响相邻的分段。

比如要求C1连续性，意味着连接点两侧的控制点是相互耦合的关系。修改这一段曲线，一定会对相邻段产生影响；如果还要保证更多的连续性，就会连带影响更多分段。

因此贝塞尔曲线的控制虽然比全局多项式更局部，但分段之间依然存在耦合，修改一段会影响至少相邻两段。

这就形成了一组矛盾：增加曲线的阶数和连续性，会提升光滑性，但同时也会增加全局性，削弱局部控制能力。

总的来说，三阶贝塞尔曲线是最常用的，更高阶计算会更复杂。贝塞尔曲线被广泛的画图工具包所支持，比如我们课程实验使用的OpenGL，就原生支持贝塞尔曲线。

前沿拓展：犹他茶壶与G2连续问题

贝塞尔曲线从1960年代出现至今，已经是非常经典的内容。但有趣的是，近年的SIGGRAPH国际会议上，依然有相关的研究工作。

过去大家普遍认为，贝塞尔曲线能保证C1连续，保证不了G2连续是可以接受的。但最近的一篇SIGGRAPH论文专门展示了：只有C1连续性、没有G2连续性，确实会带来可见的问题。

这项工作和犹他茶壶（Utah Teapot）50周年相关。犹他茶壶是较早的图形学3D数字化模型，基于贝塞尔曲面设计。当时的设计者手工调整了一套贝塞尔曲面参数，来匹配真实茶壶的外形。为了保证连续性，最终得到的贝塞尔曲面茶壶是C1连续的，它被广泛应用在各类图形学渲染任务中。

渲染中的反射现象，会放法线连续性的问题。如果法线足够连续，反射出的窗户依然是横平竖直的；如果法线不够连续，窗户的投影就会被扭曲。

大家可以去论文的网站上看动态效果，会有更直观的感受：茶壶曲面上有一条分界线，能明显看到窗户反射有时被“吃掉”、有时又出现，这就是法线变化速率不连续、C1连续性不足带来的视觉问题。而达到G2连续的曲面，反射效果会更加光滑。

如果想让曲线从C1变成G2，就不再是三阶贝塞尔曲线了。可以通过增加阶数的方式，让曲率达到更连续的状态；如果不想增加阶数，就必须分更多的段，来满足更高的连续性需求。

这项工作的核心贡献是：在提高连续性的同时，依然能让用户像控制贝塞尔曲线一样控制曲线。

其他参数化曲线

既然贝塞尔曲线在高阶连续性上存在局限，我们自然会问：能不能找到一种曲线，更容易满足G2甚至C2连续性，同时又方便用户控制，不需要反复调整左右两边？

接下来我们介绍其他几类曲线。

三次样条曲线

样条曲线（spline curve）是真实世界中天然存在的一类曲线。比如木匠制作木条时，敲几个控制点，把有韧性的木条或竹条放在中间，木条会自动平衡内部张力，稳定平衡后的形状，就是自然三次样条曲线的效果。同理，弹性绳固定几个点后，自然张成的曲线也是样条曲线。

我们先对曲线做一个分类：

- **插值曲线**：精确经过给定的控制点，对位置的控制更精准；
- **近似曲线**：不精确经过所有控制点，只在大致区域内，给曲线留出更多自由度来保证光滑性。

贝塞尔曲线就属于近似曲线，每一段内部都在控制点的包围盒内。插值曲线有可能超出包围盒，但至少已知几个点的固定位置，空间范围依然可计算。

我们来看三次插值样条曲线，这也是产业界用得比较多的曲线。它本身是分段的，每一段都是三次多项式，通过连接构成整体曲线。我们通常希望它至少满足C0连续，最好C1也连续。

接下来的问题是：如果已知一组控制点，要求曲线是三次插值曲线，同时满足C2连续性，这条曲线能不能求出来？

答案是可以的。每一段三次多项式有4个参数，两段就有8个参数。我们可以通过约束条件列方程：

- 端点经过控制点，得到位置约束方程；

- C1连续，得到一阶导数约束方程；
- C2连续，得到二阶导数约束方程。

把所有边界条件列出来，会得到一个方程组。对于两段三次曲线，未知数和约束条件的数量正好相等，因此可以求解出所有参数。

最终得到的就是自然三次样条曲线，它天然满足二阶连续的需求。

它之所以能和自然界的现象匹配，是因为二阶导连续的情况下，曲线能够最小化弹性势能，是物体满足约束条件下能量最小的状态。

但它也有不足：求解过程是全局解方程，改变其中一个控制点，就会改变所有约束，最终所有分段的参数都会发生变化。也就是说，修改一个点，整条曲线都会跟着变动，来保证整体的C2连续性。

这再次体现了连续性和局部控制之间的矛盾：连续性越高，控制的全局性就越强。三次样条非常光滑，但局部控制能力差。

Catmull-Rom曲线与厄米曲线

在设计领域，很多时候大家会选择放弃部分连续性，比如只要求C1连续，来换取更强的局部控制能力——修改某一段曲线时，其他分段不受影响。

这类曲线的代表就是**Catmull-Rom曲线**和**厄米（Hermite）曲线**，都属于C1连续的曲线，在图形学中应用广泛。

Catmull-Rom曲线

Catmull-Rom曲线的定义是：曲线本身连续，并且中间点的一阶微分，等于前后两个控制点连线的方向。

基于这两个约束，通过当前点、前一个点和后一个点的位置，就可以唯一解算出每一段三次多项式的具体形式。也就是说，每一段曲线只受相邻三个点的影响，实现了局部控制。

厄米曲线

厄米曲线则是把一阶微分的自由度直接交给用户：由用户决定每个端点的位置，以及每个控制点上的一阶微分。

点的位置和一阶微分构成四个约束条件，唯一控制一段曲线的多项式表达。每一段曲线都由自己的端点和微分控制，用户只要保证相邻端点的微分一致，就能满足C1连续。

厄米曲线和贝塞尔曲线之间存在一一对应的映射关系：任何一条厄米曲线，都有一条完全对应的贝塞尔曲线。

原因在于：厄米曲线给定了两个端点和两端的一阶微分，而三阶贝塞尔曲线由两个端点和两个控制点定义。已知端点和一阶微分，就可以计算出对应的控制点位置，得到完全相同的曲线。反过来，给定贝塞尔曲线的控制点，也可以算出端点处的一阶微分，得到对应的厄米曲线。因此二者的表达能力是一一对应的，厄米曲线可以看作是贝塞尔曲线中满足C1连续约束的一个子集。

B样条曲线简介

最后我们简单介绍B样条曲线，下节课会详细讲解。B样条的B是basis（基）的意思。

B样条曲线有两个核心特点：

1. 操作是局部影响的，修改一个控制点只会影响附近的分段；
2. 可以自动满足甚至C2的连续性。

B样条曲线属于近似曲线，它不经过任何控制点，但一定保证曲线在控制点的包围盒内部。通过减少对精确位置的限制，B样条曲线获得了更好的光滑性，具备非常多优秀的特性。

B样条曲线和贝塞尔曲线有很多相似性。可以这么理解：把两条贝塞尔曲线连接的公共端点拿掉，在相邻的控制点之间做插值，离哪个控制点近，哪条曲线的权重就高，最终过渡得到的就是B样条曲线。

下节课我们会详细拆解B样条曲线。

课堂问答

问题1：C1连续和G1连续的核心区别是什么？

答：核心区别在于是否关心参数变化的“速度”。

C1连续是对参数 t 的一阶导数连续，既要求切线方向相同，也要求导数的大小相等，对应参数 t 变化的速率一致。如果把曲线想象成小球运动的轨迹，C1连续的情况下小球会以均匀的速度经过连接点。

G1连续是几何连续性，只要求切线方向一致，不关心导数的大小，也就是不关心参数变化的快慢。哪怕两边参数变化的速度不同，只要切线方向相同，就满足G1连续。

比如两条曲线在连接点处切线方向一致，但一边参数变化快、一边变化慢，几何上看起来是相切的，这就是G1连续，但不是C1连续。

问题2：厄米曲线和贝塞尔曲线的对应关系是什么？

答：任何一条厄米曲线，都可以找到一条完全对应的贝塞尔曲线，二者表达能力是一一对应的。

厄米曲线由两个端点和两端的一阶微分定义，而三阶贝塞尔曲线由两个端点和两个控制点定义。已知端点和一阶微分，就可以计算出对应的控制点位置，得到完全相同的曲线。反过来，给定贝塞尔曲线的控制点，也可以算出端点处的一阶微分，得到对应的厄米曲线。

厄米曲线可以看作是贝塞尔曲线中，满足C1连续约束的一个子集。

问题3：伯恩斯坦多项式可以表示任意多项式曲线吗？

答：是的。伯恩斯坦多项式本身构成了多项式空间的一组基，因此它可以表示出任意同阶的多项式曲线。这也是为什么各类三次曲线之间可以相互转换的原因。

问题4：考试中曲线部分的考察重点是什么？

答：考试中曲线部分的考察不会特别难，核心是希望大家建立显式、隐式和参数化曲线的概念与直观感受。习题中会涉及相关实现，考试不会考过于细节的区别。